



UNAH
UNIVERSIDAD NACIONAL
AUTÓNOMA DE HONDURAS

Campus
Choluteca



UNIVERSIDAD NACIONAL AUTÓNOMA DE HONDURAS

CAMPUS CHOLUTECA

**CENTRO UNIVERSITARIO REGIONAL DEL LITORAL
PACÍFICO**

PROYECTO FINAL SISTEMAS OPERATIVOS II

Integrantes:

Allan Mauricio Ramirez Rodriguez - 20232330124

Amilcar Yovany Aguilar Ponce - 20242300085

Angel David Hernández Alvarez - 20242300031

Jafet Armando Alcantara Portillo - 202323001199

Olber Ssamir Nuñez Izaguirre - 20232300089

Asignatura:

Sistemas Operativos II ISC334

Catedrático:

Ph.D. Wilson Octavio Villanueva Castillo

Choluteca, 4 de julio del 2026



INDICE

INTRODUCCIÓN	9
Script: deploy-unah-conecta.sh — Despliegue integral de UNAH Conecta (WordPress "todo en uno")23	
Script 0: Configuración de IP Estática (Netplan)	39
Qué hace en general	39
Cómo funciona paso a paso	39
Requisitos y dependencias asumidas	39
Resultado final	39
Bloques de código más importantes	40
Script 1: 01-update.sh	41
Qué hace en general	41
Cómo funciona paso a paso	41
Requisitos y dependencias asumidas	42
Resultado final	42
Bloques de código más importantes	42
Script 2: 02-apache.sh	45
Qué hace en general	45
Cómo funciona paso a paso	45
Requisitos y dependencias asumidas	45



Resultado final	45
Bloques de código más importantes	46
Qué hace en general	47
Cómo funciona paso a paso	47
Requisitos y dependencias asumidas	47
Resultado final	48
Bloques de código más importantes	48
Script 4: 04-php.sh	50
Qué hace en general	50
Cómo funciona paso a paso	50
Requisitos y dependencias asumidas	50
Resultado final	50
Bloques de código más importantes	51
Qué hace en general	53
Cómo funciona paso a paso	53
Requisitos y dependencias asumidas	54
Resultado final	54
Bloques de código más importantes	54
Qué hace en general	56



Cómo funciona paso a paso	56
Requisitos y dependencias asumidas	57
Resultado final	57
Bloques de código más importantes	58
Qué hace en general	60
Cómo funciona paso a paso	60
Requisitos y dependencias asumidas	61
Resultado final	61
Bloques de código más importantes	61
Script 8: 08-security.sh	64
Qué hace en general	64
Cómo funciona paso a paso	64
Requisitos y dependencias asumidas	65
Resultado final	65
Bloques de código más importantes	65
09-backup.sh	68
Qué hace en general	68



Genera respaldos completos e integrales de las bases de datos (MariaDB) y de los componentes de archivos esenciales (wp-content y moodledata), aplicando además una política de retención para eliminar copias antiguas.	68
Cómo funciona paso a paso	68
Paso 1 — Generación de contexto: Genera un identificador único basado en fecha/hora (TIMESTAMP=\$(date +%Y%m%d_%H%M%S)) y prepara la carpeta objetivo con permisos estictos 700.	68
Paso 2 — Respalos de MySQL/MariaDB: Ejecuta mysqldump con los parámetros --single-transaction --quick para WordPress y Moodle, verifica que los volcados sean válidos y los comprime con gzip.	68
Paso 3 — Embalaje de archivos: Empaqueta las carpetas wp-content y moodledata mediante tar -czf de forma relativa y prueba la integridad de los archivos resultantes con tar -tzf.	68
Paso 4 — Informe de utilización: Despliega el peso final en disco de cada archivo generado con la utilidad du -h.	68
Paso 5 — Purga automática por retención: Utiliza el comando find con el parámetro -mtime +"\$BACKUP_RETENTION_DIAS" para localizar y eliminar archivos de respaldo que superen la cantidad de días permitidos.	68
Requisitos y dependencias asumidas	68
Acceso administrativo a MySQL/MariaDB, utilidades mysqldump, gzip, tar, find, y espacio libre suficiente en el disco.	68
Resultado final	68



Un conjunto de 4 archivos de copia de seguridad validados, garantizando la continuidad del negocio y el control del espacio en disco.....	68
Bloques de código más importantes:.....	68
Bash.....	69
ARCHIVOS_A_ELIMINAR=\$(find "\$BACKUP_DIR" -type f -mtime +"\$BACKUP_RETENTION_DIAS" \.....	69
\(-name "wp_db_*.sql.gz" -o -name "moodle_db_*.sql.gz" -o -name "wp_content_*.tar.gz" -o -name "moodledata_*.tar.gz" \) 2>/dev/null true).....	69
CANTIDAD_ELIMINADOS=0	69
if [[-n "\$ARCHIVOS_A_ELIMINAR"]]; then.....	69
CANTIDAD_ELIMINADOS=\$(echo "\$ARCHIVOS_A_ELIMINAR" grep -v '^\$' wc -l echo 0).....	69
echo "\$ARCHIVOS_A_ELIMINAR" xargs rm -f error "Error al eliminar respaldos antiguos."	69
fi.....	69
Script 10: deploy-all.sh	69
Qué hace en general	69
Cómo funciona paso a paso	70
Requisitos y dependencias asumidas.....	70
Resultado final	70
Bloques de código más importantes	71



INTRODUCCIÓN

En la actualidad, la administración de servidores Linux y la automatización de tareas constituyen competencias fundamentales para los profesionales del área de las tecnologías de la información. Las organizaciones demandan infraestructuras capaces de ofrecer servicios confiables, seguros y escalables, reduciendo al mismo tiempo el tiempo de implementación y la posibilidad de errores derivados de configuraciones manuales. En este contexto, la automatización mediante Bash Scripting se ha convertido en una herramienta indispensable para estandarizar procesos, facilitar el mantenimiento de los sistemas y garantizar que una infraestructura pueda ser desplegada de forma rápida y reproducible.

El presente proyecto tiene como finalidad diseñar e implementar una infraestructura de servidor basada en **Ubuntu Server LTS**, automatizando completamente la instalación, configuración y puesta en funcionamiento de los diferentes servicios requeridos para el proyecto **UNAH-CONNECTA**. Para lograr este objetivo se desarrolló un conjunto de scripts independientes que permiten instalar y configurar los componentes esenciales del servidor, entre ellos Apache2 como servidor web, MariaDB como sistema gestor de bases de datos, PHP y sus extensiones necesarias, las plataformas WordPress y Moodle, así como servicios complementarios de administración, seguridad, acceso remoto, transferencia de archivos, monitoreo y respaldo.

Uno de los principios fundamentales del proyecto consiste en aplicar una arquitectura modular, donde cada componente del sistema sea instalado mediante un script específico. Este enfoque permite que cualquier administrador pueda reconstruir completamente el servidor ejecutando los scripts en el orden establecido, facilitando la reutilización del código, el mantenimiento de la infraestructura y la implementación de futuras mejoras sin afectar el funcionamiento general del sistema. Asimismo, el modularidad favorece la detección y corrección de errores, permitiendo intervenir únicamente sobre el componente afectado sin necesidad de reinstalar toda la plataforma.

Como parte de la infraestructura implementada, se configuró un servidor web capaz de alojar simultáneamente un sitio institucional desarrollado en WordPress y una plataforma educativa Moodle mediante el uso de Apache Virtual Hosts, utilizando un único servidor Ubuntu y una única instancia de MariaDB. Además, se incorporaron mecanismos de seguridad como la configuración del firewall UFW, el acceso remoto mediante SSH, la implementación de servicios FTP y SFTP para la administración de archivos, la automatización de copias de seguridad y la instalación de Webmin como plataforma de administración remota del servidor. Estas herramientas permiten administrar de manera centralizada los diferentes servicios, monitorear el estado del sistema y facilitar las tareas de mantenimiento.



Durante el desarrollo del proyecto también se aplicaron buenas prácticas de administración de sistemas Linux, organización de scripts, gestión de permisos, documentación técnica y validación del funcionamiento de cada uno de los servicios implementados. Cada procedimiento fue probado de manera individual y posteriormente de forma integrada para garantizar la correcta interacción entre los distintos componentes del servidor, verificando la disponibilidad de los servicios web, la conectividad con las bases de datos, el acceso remoto y el correcto funcionamiento de los mecanismos de seguridad y monitoreo.

Finalmente, esta documentación presenta de forma detallada la arquitectura propuesta, los requerimientos del servidor, la explicación técnica de cada uno de los scripts desarrollados, el flujo de ejecución recomendado, los procedimientos de instalación y configuración, las pruebas realizadas, las evidencias obtenidas y las principales dificultades encontradas durante la implementación.

El propósito es que este documento sirva como una guía técnica completa que permita comprender el funcionamiento de la solución implementada y posibilite la reconstrucción del servidor siguiendo únicamente los procedimientos aquí descritos, cumpliendo así con los objetivos establecidos para el proyecto final de la asignatura Sistemas Operativos II.

Objetivos

Objetivo General

Diseñar e implementar una infraestructura de servidor completamente automatizada mediante Bash Scripting sobre Ubuntu Server LTS, que permita desplegar y configurar los servicios necesarios para el proyecto UNAH-CONNECTA, incluyendo Apache2, MariaDB, PHP, WordPress, Moodle, Virtual Hosts, Webmin, servicios de transferencia de archivos y mecanismos de seguridad, garantizando un entorno funcional, organizado, seguro y fácilmente reproducible.

Objetivos Específicos

- Automatizar la actualización del sistema operativo Ubuntu Server y la instalación de los paquetes básicos requeridos para el funcionamiento del servidor.
- Instalar y configurar la pila LAMP (Linux, Apache2, MariaDB y PHP) mediante scripts independientes que permitan una implementación rápida y estandarizada.
- Configurar un servidor MariaDB creando bases de datos, usuarios y privilegios independientes para las plataformas WordPress y Moodle.
- Implementar y configurar WordPress como portal institucional y Moodle como plataforma educativa, asegurando el funcionamiento simultáneo de ambas aplicaciones mediante Apache Virtual Hosts.
- Configurar servicios de transferencia de archivos mediante FTP y SFTP para facilitar la administración remota y segura del contenido del servidor.
- Implementar mecanismos de seguridad mediante la configuración del firewall UFW, el acceso remoto por SSH y la asignación adecuada de permisos sobre archivos y directorios del sistema.
- Automatizar la instalación y configuración de Webmin para centralizar la administración del servidor, permitiendo el monitoreo de recursos, la gestión de servicios, usuarios, procesos y tareas programadas.
- Desarrollar scripts de respaldo y monitoreo que permitan generar copias de seguridad de la información, registrar el estado de los servicios y facilitar las tareas de mantenimiento preventivo del servidor.



- Documentar detalladamente el proceso de instalación, configuración, ejecución y validación de cada uno de los scripts desarrollados, permitiendo que cualquier administrador pueda reconstruir completamente la infraestructura siguiendo los procedimientos establecidos.
- Verificar el correcto funcionamiento de todos los servicios implementados mediante pruebas técnicas y evidencias que demuestren la disponibilidad, estabilidad e integración de cada componente de la infraestructura.

MARCO TEÓRICO

El marco teórico constituye el sustento conceptual sobre el cual se desarrolla la infraestructura de servicios del proyecto. En esta sección se analizan las tecnologías, protocolos y arquitecturas involucradas, justificando su implementación dentro de un entorno de servidores sobre GNU/Linux.

1 Ubuntu Server

Ubuntu Server es una distribución del sistema operativo GNU/Linux orientada a entornos corporativos y de infraestructura, desarrollada por Canonical. A diferencia de las versiones *Desktop*, prescinde por defecto de un entorno gráfico de usuario (GUI) para priorizar el rendimiento, la eficiencia en el uso de recursos de hardware y la estabilidad del sistema.

Principios Fundamentales y Arquitectura:

- **Kernel de Linux:** Administra la comunicación entre el hardware del servidor (CPU, RAM, discos, interfaces de red) y las aplicaciones o servicios del sistema. Ofrece un alto rendimiento en multitarea y soporte avanzado de redes.
- **Modelo LTS (Long Term Support):** Las versiones de soporte a largo plazo ofrecen actualizaciones de seguridad y mantenimiento durante un periodo extendido (5 años estándar), garantizando estabilidad operacional sin riesgos de discontinuidad.
- **Gestión de Paquetes (apt y dpkg):** Utiliza el sistema de gestión de paquetes de Debian. El comando apt (Advanced Package Tool) facilita la instalación, actualización y resolución de dependencias de software desde repositorios oficiales y verificados.
- **Consola de Comandos (CLI):** La interacción se realiza íntegramente mediante la interfaz de línea de comandos, lo que reduce la superficie de ataque, el consumo de memoria RAM y el uso de ciclo de CPU.

2 Bash (Bourne Again Shell)

Bash es un intérprete de comandos y lenguaje de programación de Shell desarrollado para el proyecto GNU. Funciona como la interfaz primaria de comunicación entre el usuario y el sistema operativo en la mayoría de las distribuciones Linux.

Integración en el Proyecto y Automatización:

- **Ejecución de Mandatos:** Interpreta y procesa comandos interactivos ingresados en la terminal, así como scripts estructurados.
- **Scripting y Automatización:** Permite escribir scripts ejecutables (con extensión .sh) para automatizar tareas complejas y repetitivas, tales como aprovisionamiento de servicios, configuración de usuarios, respaldos de información y monitoreo de recursos.
- **Estructuras de Control y Lógica:**
 - **Variables:** Almacenamiento de rutas, credenciales temporales y configuraciones de entorno.
 - **Estructuras Condicionales (if, case):** Evaluación de códigos de salida (\$?) para verificar si un paquete se instaló correctamente o si un puerto está abierto.
 - **Bucles (for, while):** Procesamiento iterativo de listas de usuarios, rutas de archivos o tareas programadas.
 - **Redirección de flujos (>, >>, 2>, |):** Encadenamiento de la salida estándar y de error hacia archivos de log o entre diferentes comandos mediante tuberías (*pipes*).

3 Servidor Web HTTP Apache (httpd)

Apache HTTP Server es un servidor web modular de código abierto que procesa peticiones mediante el protocolo HTTP/HTTPS. Su función principal es recibir las solicitudes de clientes web (navegadores) y entregar el contenido estático (HTML, CSS, imágenes) o dinámico (mediante intérpretes como PHP) solicitado.

Componentes Clave y Funcionamiento:

- **Arquitectura Modular:** Carga únicamente los módulos necesarios para su funcionamiento, tales como mod_rewrite (reescritura de URLs), mod_ssl (cifrado TLS/SSL) y mod_php (ejecución de scripts PHP).
- **Directivas de Configuración:** La administración en Ubuntu Server se centraliza en el directorio /etc/apache2/. Sus archivos estructurados principales son:
 - apache2.conf: Archivo principal de configuración global.

- ports.conf: Define los puertos de escucha (e.g., puerto 80 para HTTP, 443 para HTTPS).
 - sites-available/: Contiene los archivos de configuración de cada sitio web.
 - sites-enabled/: Contiene enlaces simbólicos hacia los sitios activos.
- **Gestión del Servicio:** Operado a través del sistema de iniciación systemd mediante comandos como systemctl status apache2, systemctl reload apache2 o systemctl restart apache2.

4 MariaDB

MariaDB es un sistema de gestión de bases de datos relacionales (RDBMS) de código abierto, derivado y mantenido por la comunidad como una alternativa compatible y con mejoras de rendimiento respecto a MySQL.

Características y Estructura Técnica:

- **Modelo Relacional:** Organiza la información en tablas estructuradas mediante filas y columnas, garantizando la integridad de las relaciones a través de claves primarias y foráneas.
- **Motor de Almacenamiento InnoDB:** Soporta transacciones **ACID** (Atomicidad, Consistencia, Aislamiento y Durabilidad), garantizando que las operaciones en la base de datos se ejecuten de manera segura y sin pérdida de datos ante fallos.
- **Seguridad e Hardening:** Tras la instalación, se ejecuta la herramienta mariadb-secure-installation para eliminar tablas de prueba, restringir el acceso remoto a la cuenta root y establecer políticas de autenticación seguras.
- **Interacción:** Administrado localmente mediante la consola mariadb -u root -p o mediante interfaces remotas y scripts de automatización SQL.

5 PHP (Hypertext Preprocessor)

PHP es un lenguaje de programación interpretado diseñado para el desarrollo web de lado del servidor. Es el motor necesario para la ejecución de plataformas dinámicas como WordPress y Moodle.

Integración y Módulos:

- **Interpretación del Lado del Servidor:** Cuando un usuario solicita una página .php, Apache deriva el procesamiento al intérprete de PHP, el cual ejecuta la lógica de negocio,

consulta la base de datos MariaDB y devuelve un documento HTML plano al navegador del cliente.

- **Extensión de Módulos Específicos:** Para garantizar el funcionamiento correcto de aplicaciones web avanzadas, PHP requiere de librerías adicionales:
 - php-mysql: Comunicación nativa con bases de datos MariaDB/MySQL.
 - php-gd y php-intl: Manejo de imágenes y soporte de internacionalización.
 - php-xml, php-curl, php-zip, php-mbstring: Manipulación de formatos de datos, peticiones HTTP, archivos comprimidos y codificación multibyte.
- **Configuración (php.ini):** Permite ajustar parámetros de rendimiento y seguridad como memory_limit, upload_max_filesize, post_max_size y max_execution_time.

6 Protocolo FTP (File Transfer Protocol)

FTP es un protocolo de capa de aplicación basado en la arquitectura cliente-servidor, diseñado para la transferencia de archivos a través de redes TCP/IP.

Mecanismos de Funcionamiento:

- **Conexión de Doble Canal:**
 - **Canal de Control (Puerto 21):** Transmite los comandos de administración y autenticación (nombre de usuario y contraseña).
 - **Canal de Datos (Puerto 20 en modo activo o puertos dinámicos en modo pasivo):** Transfiere los datos reales de los archivos y los listados de directorios.
- **Implementación mediante vsftpd (Very Secure FTP Daemon):** Demonio estándar en sistemas Linux para proveer servicio FTP con un enfoque en seguridad, velocidad y bajo consumo de memoria. Su configuración se gestiona en /etc/vsftpd.conf.
- **Limitación de Seguridad:** Transmite credenciales y archivos en texto plano (sin cifrar), por lo que se recomienda restringir su uso a entornos controlados o sustituirlo por protocolos cifrados.

7 Protocolo SFTP (SSH File Transfer Protocol)

SFTP es un protocolo independiente que proporciona capacidades de acceso, transferencia y gestión de archivos de forma completamente cifrada a través de una sesión de SSH (*Secure Shell*).



Ventajas y Configuración Avanzada:

- **Cifrado de Canal:** Reutiliza la infraestructura del protocolo SSH (puerto 22) para cifrar tanto las credenciales de acceso como la totalidad de los datos transmitidos, protegiendo la información contra interceptaciones (*eavesdropping*).
- **Entorno Restringido (*Chroot Jail*):** Mediante la directiva ChrootDirectory en la configuración de SSH (`/etc/ssh/sshd_config`), se puede confinar a un usuario exclusivamente dentro de su directorio personal, impidiéndole navegar por la estructura raíz de archivos del servidor.
- **Gestión de Permisos:** Elimina la necesidad de ejecutar un servicio FTP dedicado independiente, reduciendo el número de demonios activos y el consumo de puertos abiertos en el cortafuegos.

8 WordPress

WordPress es un Sistema de Gestión de Contenidos (CMS) modular desarrollado en PHP y apoyado en bases de datos MariaDB/MySQL.

Arquitectura de Despliegue en Servidores:

- **Estructura de Archivos:**
 - `wp-config.php`: Archivo crítico donde se definen las credenciales de conexión a la base de datos, las claves de seguridad (*salts*) y el prefijo de las tablas.
 - `wp-content/`: Directorio que almacena temas, plugins y archivos multimedia cargados por los usuarios.
 - `.htaccess`: Utilizado por el módulo `mod_rewrite` de Apache para gestionar permalinks y reglas de reescritura de URLs.
- **Requisitos de Permisos y Propietarios:** El directorio raíz del sitio debe pertenecer al usuario y grupo del servidor web (`www-data:www-data`), con permisos estrictos de lectura y escritura (755 para carpetas, 644 para archivos) para evitar compromisos de seguridad.

9 Moodle (Modular Object-Oriented Dynamic Learning Environment)

Moodle es una plataforma de gestión de aprendizaje (LMS) orientada a la creación de entornos virtuales educativos.



Requisitos Técnicos y Configuración Avanzada:

- **Directorio de Datos Extendido (moodledata):** Moodle requiere de una carpeta especial fuera del alcance del directorio público del servidor web (web root) para almacenar entregas de tareas, recursos cargados, caché y perfiles de usuario de forma segura.
- **Dependencias y PHP:** Requiere un conjunto más extenso de extensiones de PHP y un ajuste elevado en los límites de memoria (memory_limit recomendado en mínimo 256M o 512M).
- **Tareas Programadas (Cronjobs):** Necesita de una tarea automatizada configurada en el planificador del sistema (crontab) ejecutando admin/cli/cron.php para el procesamiento en segundo plano de envíos de correos, mantenimiento de la base de datos y tareas del sistema.

10 Virtual Hosts (Anfitriones Virtuales)

Virtual Hosts es una técnica de configuración en Apache que permite alojar y servir múltiples sitios web o dominios independientes desde una misma instancia física o máquina virtual de servidor.

Tipos y Estructura de Configuración:

- **Basados en Nombre (Name-based Virtual Hosts):** El servidor diferencia las peticiones según la cabecera HTTP Host enviada por el navegador del cliente, dirigiendo la solicitud al directorio correcto según corresponda (wordpress.local, moodle.local, etc.).

- **Estructura de un Archivo .conf:**

Apache

```
<VirtualHost *:80>
    ServerName dominio.com
    ServerAdmin admin@dominio.com
    DocumentRoot /var/www/html/proyecto
    ErrorLog ${APACHE_LOG_DIR}/error.log
    CustomLog ${APACHE_LOG_DIR}/access.log combined
</VirtualHost>
```

- **Activación de Sitios:** Se habilitan vinculando los archivos mediante el comando `a2ensite nombre_sitio.conf` y recargando la configuración de Apache.

11 Webmin



Webmin es una herramienta de administración gráfica orientada a la web para sistemas operativos Unix/Linux.

Arquitectura y Capacidades:

- **Servidor Web Independiente:** Incluye su propio servidor de miniprocésamiento corriendo por defecto en el puerto 10000 con soporte para cifrado SSL/TLS.
- **Administración Modular:** Permite editar directamente los archivos de configuración del sistema desde la web para:
 - Creación y gestión de cuentas de usuario del sistema y grupos.
 - Control e inspección de servicios (systemd).
 - Edición de las reglas del cortafuegos UFW y tablas de enrutamiento.
 - Monitoreo gráfico de rendimiento en tiempo real.

12 UFW (Uncomplicated Firewall)

UFW es una herramienta de gestión de cortafuegos desarrollada para simplificar la configuración de las reglas de filtrado de paquetes de iptables y nftables en Linux.

Políticas y Estrategia de Seguridad:

- **Principio de Mínimo Privilegio (Default Deny):** La política estándar consiste en denegar todo el tráfico entrante (*Default Incoming Deny*) y permitir el tráfico saliente (*Default Outgoing Allow*).
- **Apertura Selectiva de Puertos e Interfaces:** Permite abrir únicamente los puertos necesarios para la operación de la infraestructura:
 - **22/TCP:** Tráfico SSH y SFTP.
 - **80/TCP & 443/TCP:** Tráfico Web HTTP y HTTPS.
 - **20, 21/TCP:** Tráfico FTP.
 - **10000/TCP:** Acceso a la interfaz de Webmin.
- **Comandos Principales:** Activación (ufw enable), verificación (ufw status verbose), e inserción de reglas (ufw allow 80/tcp).

13 SSH (Secure Shell)

SSH es un protocolo de red y conjunto de utilidades que permite acceder y administrar de forma segura a máquinas remotas a través de un canal de comunicación cifrado.

Protocolos de Autenticación y Hardening:

- **Cifrado de la Sesión:** Emplea criptografía asimétrica para la negociación inicial de claves y cifrado simétrico para la transmisión de datos en la sesión activa.
- **Mecanismos de Autenticación:**
 - **Por Contraseña:** Verificación mediante el sistema PAM de Linux.
 - **Por Claves Públicas/Privadas:** Uso del par de llaves RSA o Ed25519, permitiendo un acceso más seguro y eliminando los riesgos de ataques por fuerza bruta sobre contraseñas.
- **Endurecimiento del Servicio (/etc/ssh/sshd_config):** Prácticas recomendadas que incluyen la desactivación del acceso directo para el usuario root (PermitRootLogin no), cambio del puerto estándar, y la deshabilitación del reenvío de X11 (X11Forwarding no).

ARQUITECTURA

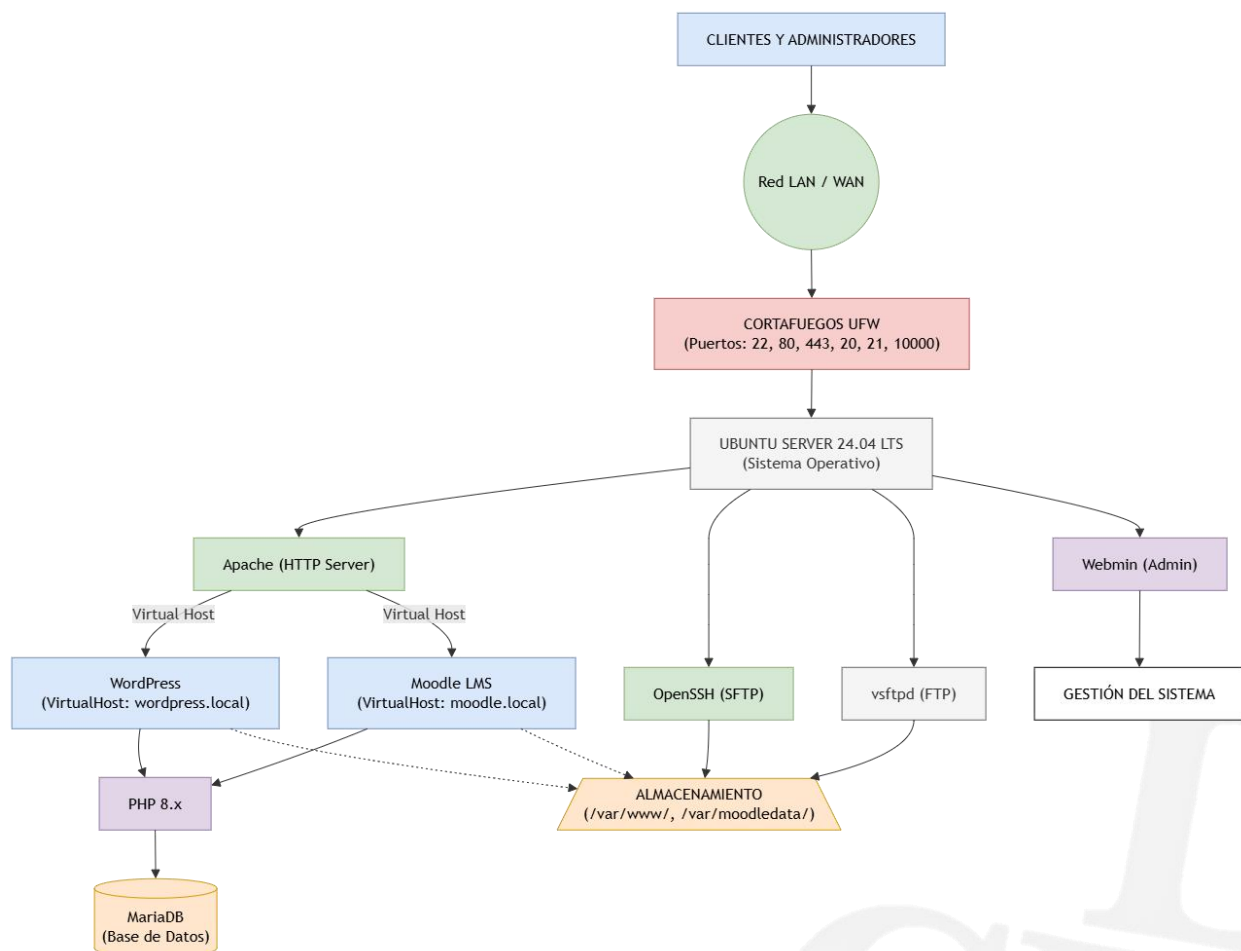
La arquitectura del sistema describe el diseño lógico y físico de la infraestructura desplegada sobre **Ubuntu Server**. Se detallan los componentes de red, las capas de software, los flujos de comunicación y las interacciones entre los usuarios finales, los administradores y los servicios alojados.

1 Diagrama de Infraestructura

Tenemos un diagrama que representa la infraestructura instalada y funcionando en nuestro proyecto.

Podemos ver como cada parte del proyecto se conecta y se necesita entre si y como es que el usuario llega a conectarse a el servidor.

Esquema Conceptual del Sistema:





2 Descripción de la Arquitectura

El diseño arquitectónico adoptado se basa en un modelo **multicapa desacoplado**, compuesto por las siguientes capas operativas:

2.1 Capa de Seguridad y Perímetro (Firewall)

UFW (Uncomplicated Firewall): Actúa como la primera línea de defensa del servidor. Filtra las peticiones entrantes según reglas predefinidas, bloqueando puertos no autorizados y permitiendo únicamente el tráfico estrictamente necesario para la operación de la plataforma (HTTP, FTP, SFTP, SSH y Webmin).

2.2 Capa de Presentación y Servidor Web (Web Tier)

Servidor HTTP Apache: Es el motor principal que recibe todas las peticiones web en el puerto 80.

Módulo Virtual Hosts: Permite segmentar el tráfico web entrante. Según el nombre de dominio utilizado por el cliente (wordpress.local o moodle.local), Apache redirige la solicitud hacia su correspondiente directorio raíz (*DocumentRoot*) dentro de */var/www/*.

2.3 Capa de Aplicación y Procesamiento Dinámico (Application Tier)

Intérprete PHP: Apache deriva las peticiones con contenido ejecutable (.php) hacia el intérprete PHP y sus módulos auxiliares. Esta capa ejecuta la lógica de negocio de las aplicaciones **WordPress y Moodle**.

2.4 Capa de Datos (Data Tier)

MariaDB Database Server: Almacena la información persistente del sistema. Mantiene esquemas de bases de datos aislados y usuarios independientes con permisos restringidos para WordPress y Moodle. La comunicación entre la capa de aplicación (PHP) y la base de datos se realiza localmente a través de *sockets* o conexiones de red interna (*loopback* 127.0.0.1).

4.2.5 Capa de Transferencia de Archivos y Almacenamiento

vsftpd (FTP) & OpenSSH (SFTP): Proporcionan servicios de gestión y subida de archivos al servidor.

Los usuarios aislados (*chroot*) pueden transferir contenido web directamente a las carpetas correspondientes en */var/www/* o a sus directorios personales.

2.6 Capa de Administración y Monitoreo

Webmin & SSH (CLI): Proveen acceso administrativo a la infraestructura.



OpenSSH: Permite a los administradores gestionar el servidor por línea de comandos mediante scripts automatizados en **Bash**.

Webmin: Ofrece una interfaz web segura en el puerto 10000 para el monitoreo de recursos del sistema (CPU, RAM, uso de disco) y administración de servicios.

CONFIGURACION DE WORDPRESS

Script: `deploy-unah-conecta.sh` — Despliegue integral de UNAH Conecta (WordPress "todo en uno")

Qué hace en general

Este script es distinto a los anteriores: en vez de ser un módulo aislado dentro de una secuencia orquestada (como los scripts 01-08 de UNAH-CONECTA), es un **script monolítico de despliegue rápido** que instala la pila completa LAMP (Linux, Apache, MariaDB, PHP), descarga e instala WordPress, y además **inyecta un tema institucional personalizado** con lógica propia de matrícula, un dashboard estudiantil y usuarios de prueba, todo en una sola ejecución pensada para un entorno como AWS (detecta la IP pública automáticamente).

Cómo funciona paso a paso

1. **[1/8] Instalación de la pila LAMP:** Actualiza los repositorios (`apt update`) e instala en un solo comando Apache2, MariaDB, PHP con `libapache2-mod-php` (el módulo clásico de PHP embebido en Apache, no PHP-FPM) y el conector `php-mysql`, junto con utilidades básicas (`curl`, `wget`, `unzip`).
2. **[2/8] Configuración de la base de datos:** Crea la base `unah_db` y el usuario `unah_user` con una contraseña fija escrita directamente en el script, y le otorga todos los privilegios sobre esa base.
3. **[3/8] Descarga de WordPress:** Descarga el `.tar.gz` oficial de WordPress en `/tmp`, lo descomprime, **borra por completo el contenido existente de `/var/www/html`** (`rm -rf /var/www/html/*`) y copia ahí los archivos de WordPress recién descomprimidos.
4. **[4/8] Generación de `wp-config.php`:** Copia el archivo de ejemplo `wp-config-sample.php` a `wp-config.php` y usa `sed` para reemplazar los marcadores de nombre de base, usuario y contraseña por los valores reales. Además, inyecta con `sed` dos líneas de PHP (`WP_HOME` y `WP_SITEURL`) justo antes de la línea `define('ABSPATH'...)`, para que WordPress detecte dinámicamente el host desde el que se accede (`$_SERVER['HTTP_HOST']`), en vez de depender de una URL fija.



5. **[5/8] Inyección del tema institucional:** Crea la carpeta del tema unah-conecta-theme y genera, con dos bloques heredoc, sus dos archivos principales:
 1. style.css: solo contiene el encabezado de metadatos que WordPress exige para reconocer un tema (nombre, autor, versión).
 2. functions.php: contiene toda la lógica funcional del portal, con varias funciones enganchadas a hooks de WordPress (ver detalle en los bloques de código más abajo).
6. **[6/8] Instalación silenciosa de WordPress vía WP-CLI:** Descarga WP-CLI de la misma fuente oficial vista en scripts anteriores, y en vez de tomar el dominio de una variable de configuración, obtiene la IP pública real del servidor consultando el servicio externo checkip.amazonaws.com. Con esa IP arma la URL del sitio y ejecuta wp core install de forma no interactiva.
7. **[7/8] Activación de tema y usuarios de prueba:** Activa el tema recién creado (wp theme activate), crea un rol personalizado alumno clonado del rol nativo subscriber, crea dos usuarios de prueba con ese rol, y crea dos páginas de WordPress cuyo contenido son los shortcodes definidos en functions.php ([unah_dashboard] y [unah_matriculas]).
8. **[8/8] Permisos y arranque de servicios:** Aplica propietario www-data y permisos 755 de forma recursiva sobre todo /var/www/html, y reinicia y habilita Apache2 y MariaDB.
9. **Mensaje final:** Imprime en pantalla un resumen con la URL del sitio y, en texto plano, las credenciales del administrador y del alumno de prueba.

Requisitos y dependencias asumidas

- Ubuntu Server con acceso a internet, y en particular acceso saliente a checkip.amazonaws.com (pensado específicamente para una instancia EC2 de AWS, aunque funcionaría en cualquier servidor con salida a internet).
- Debe ejecutarse como root (usa --allow-root en cada llamada a WP-CLI, pero no valida explícitamente al inicio que el usuario sea root, a diferencia de los scripts modulares que usan require_root).



- No depende de helpers.sh ni config.env: es completamente autónomo, con todos los valores (contraseñas, nombres, textos) escritos directamente en el cuerpo del script.

Resultado final

Un sitio WordPress completamente funcional y accesible por la IP pública del servidor, con un tema institucional propio, dos páginas ya publicadas con formularios funcionales de matrícula y un dashboard estudiantil, un rol de "alumno" separado de los roles nativos de WordPress, y dos cuentas de prueba listas para validar el flujo completo sin configuración manual adicional.

Bloques de código más importantes

bash

```
rm -rf /var/www/html/*cp -R wordpress/* /var/www/html/
```

Esta línea es la más delicada del script en términos de seguridad operativa: borra sin confirmación ni respaldo previo todo el contenido existente en la raíz web. A diferencia de 05-wordpress.sh (el script modular), que primero verificaba si ya existía una instalación y abortaba para no sobrescribir datos, aquí no hay ningún control de idempotencia: ejecutar este script dos veces destruye por completo cualquier instalación anterior, incluidos archivos subidos por usuarios o configuraciones previas.

bash

```
sed -i "s/database_name_here/unah_db/g" wp-config.phpsed -i "s/username_here/unah_user/g" wp-config.phpsed -i "s/password_here/Unah2026_Secure!/g" wp-config.phpsed -i "/define( 'ABSPATH'/i define('WP_HOME', 'http://' .\$_SERVER['HTTP_HOST']);\ndefine('WP_SITEURL', 'http://' .\$_SERVER['HTTP_HOST']);" wp-config.php
```

En vez de usar wp config create (como hacía el script modular 05-wordpress.sh), aquí se genera wp-config.php editando directamente el archivo de ejemplo con sed, reemplazando los marcadores de texto que WordPress deja como plantilla. La última línea es la más interesante técnicamente: usa la opción -i de sed (insertar antes de una línea que coincide con un patrón, en este caso define('ABSPATH')) para inyectar código PHP que hace que el sitio determine su propia URL dinámicamente a partir del encabezado Host de cada petición, en lugar de tener una URL fija grabada en la base de datos. Esto es útil en AWS porque la IP pública de una instancia puede cambiar entre reinicios.

php

```
$tabla = 'Matriculas_Demo'; $wpdb->query("CREATE TABLE IF NOT EXISTS $tabla (id INT  
AUTO_INCREMENT PRIMARY KEY, id_usuario INT NOT NULL, codigo_curso  
VARCHAR(50) NOT NULL, estado_matricula VARCHAR(20), fecha_inscripcion  
DATETIME)"); $insertado = $wpdb->insert($tabla, array('id_usuario' => $id_usuario,  
'codigo_curso' => $codigo_curso, 'estado_matricula' => 'Activa', 'fecha_inscripcion' =>  
current_time('mysql')));
```

Este fragmento, dentro de functions.php, muestra que el script no solo instala WordPress "de fábrica": crea también una tabla propia (Matriculas_Demo) directamente en la base de datos de WordPress usando el objeto global \$wpdb (el wrapper de acceso a base de datos que WordPress expone a los temas y plugins). La tabla se crea de forma perezosa (CREATE TABLE IF NOT EXISTS) la primera vez que alguien matricula un curso, en lugar de crearse durante la instalación. Es una forma rápida de simular un módulo de matrícula, aunque sin las validaciones ni la separación de capas que tendría un módulo formal (por ejemplo, no valida que codigo_curso corresponda a un curso real, ni usa \$wpdb->prepare() para sanitizar la sentencia SQL de creación de tabla).

php

```
add_shortcode('unah_matriculas', 'unah_backend_matriculas_logic');  
...add_shortcode('unah_dashboard', 'unah_dashboard_estudiante_logic');
```

Estas dos líneas son las que conectan la lógica PHP escrita en functions.php con el contenido editable de WordPress: registran **shortcodes**, es decir, etiquetas cortas ([unah_matriculas], [unah_dashboard]) que un editor de páginas puede insertar en cualquier contenido, y que WordPress reemplaza en tiempo real por el HTML que devuelve la función asociada. Esto es exactamente lo que usa el paso [7/8] del script cuando crea las páginas "Dashboard Estudiantil" y "Matrícula" con wp post create.

php

```
function unah_login_redirect($redirect_to, $request, $user) {  
    if (isset($user->roles) && is_array($user->roles) && in_array('alumno', $user->roles)) {  
        return home_url();  
    }  
    return $redirect_to;} add_filter('login_redirect', 'unah_login_redirect', 99,  
3); add_filter('show_admin_bar', '__return_false');
```

Este bloque personaliza la experiencia de inicio de sesión según el rol del usuario: usa el filtro login_redirect de WordPress para interceptar a dónde se envía a un usuario después de autenticarse, y si su rol es alumno, lo redirige a la portada del sitio en vez de al panel de administración (wp-admin), que es el destino por defecto de WordPress. La segunda línea



(show_admin_bar, __return_false) oculta la barra de administración superior para todos los usuarios, no solo para los alumnos, lo cual simplifica la interfaz pero también la oculta para el propio administrador cuando navega el sitio como visitante.

```
bash
IP_ACTUAL=$(curl -s http://checkip.amazonaws.com)
wp core install \
  --url="http://$IP_ACTUAL" \
  ...
```

Esta es la pieza que hace al script "consciente de la nube": en lugar de tomar el dominio de una variable de configuración fija (como hacía 05-wordpress.sh con DOMAIN_WP), consulta en tiempo real un servicio HTTP externo de Amazon que devuelve la IP pública desde la que se origina la petición, y usa esa IP directamente como URL del sitio. Esto permite que el mismo script funcione sin modificaciones en cualquier instancia EC2 nueva, pero también acopla el script a la disponibilidad de ese servicio externo y a que el tráfico saliente hacia AWS esté permitido.

Como parte del despliegue y puesta en marcha del núcleo de WordPress, se ejecutó la parametrización de los datos iniciales del sitio institucional UNAH-CONECTA. Esta interfaz estándar permite definir de forma segura los parámetros principales del entorno de producción:

Título del Sitio (Site Title): Configurado oficialmente como UNAH-CONECTA, estableciendo la identidad principal que se visualiza en los navegadores y pestañas del sistema.

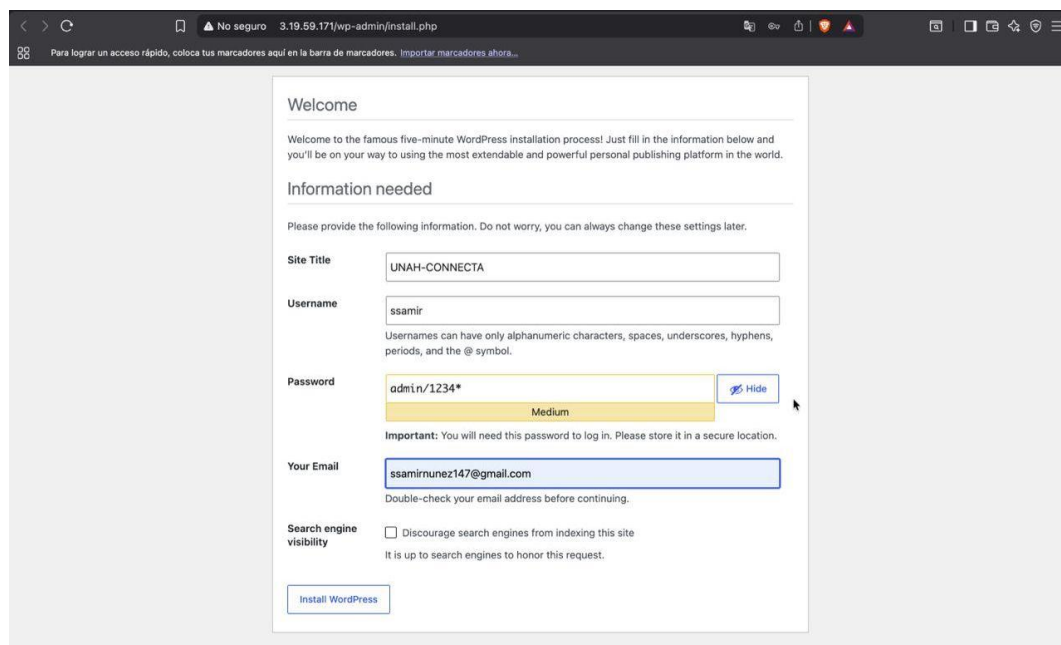
Credenciales del Administrador Principal:

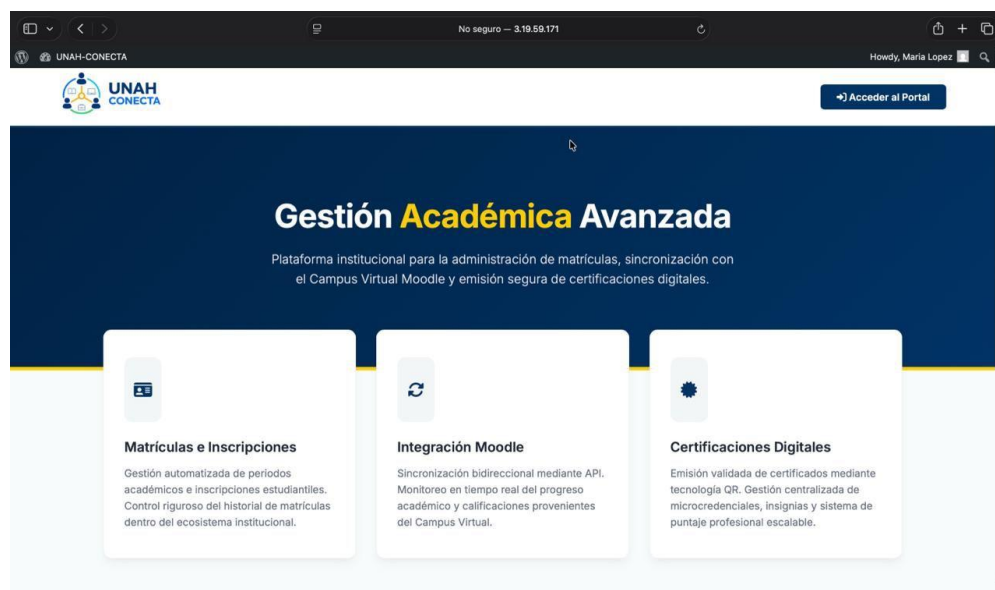
Nombre de Usuario (Username): Establecido bajo el identificador personal del responsable del despliegue (ssamir).

Contraseña (Password): Configurada con un nivel de seguridad medio-alto (admin/1234*) para la protección del acceso root al panel de control.

Correo Electrónico de Soporte (Your Email): Asignado a la cuenta institucional o de contacto del administrador (ssamirnunez147@gmail.com) para la recuperación de credenciales y avisos del sistema.

Visibilidad en Motores de Búsqueda: Se mantuvo el comportamiento predeterminado del sistema orientado a entornos de desarrollo y pruebas locales.





El diseño de la interfaz gráfica implementa la identidad institucional y presenta una estructura orientada a la Gestión Académica Avanzada, facilitando el acceso rápido a los servicios clave para la comunidad universitaria:

Cabecera Institucional: Muestra el logotipo oficial de UNAH CONECTA, acompañado en la esquina superior derecha por el indicador de sesión activa (identificando al usuario actual, ej. Maria Lopez) y un botón de acceso directo al portal.

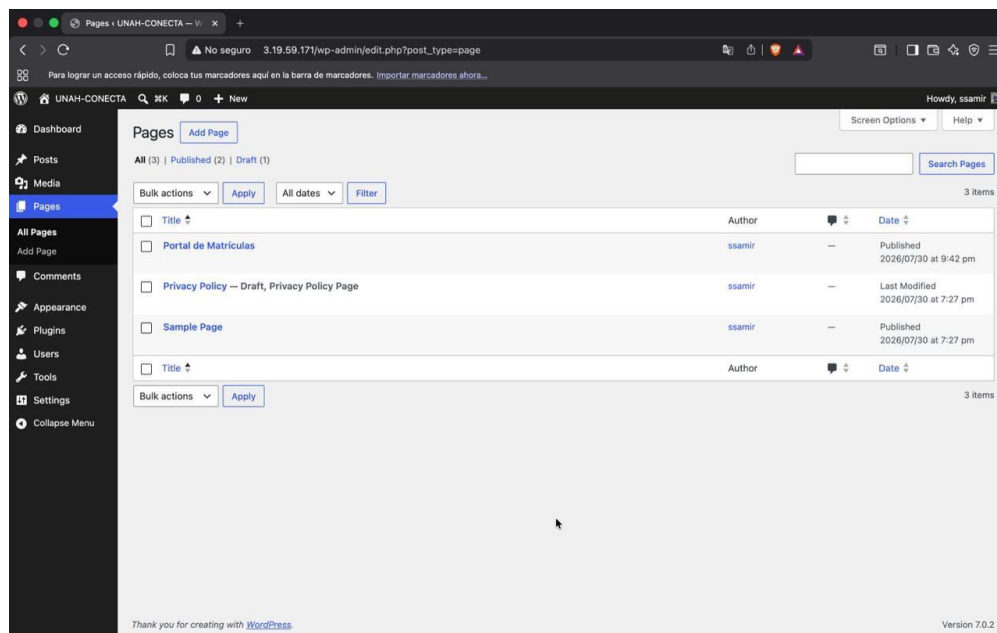
Sección Hero (Encabezado Principal): Presenta el título institucional "Gestión Académica Avanzada" junto con la descripción de la plataforma, orientada a la administración de matrículas, sincronización con el Campus Virtual Moodle y emisión de certificaciones digitales.

Módulos Interactivos de Servicios:

Matrículas e Inscripciones: Módulo para la gestión automatizada de períodos académicos, inscripciones de estudiantes y control del historial dentro del ecosistema institucional.

Integración Moodle: Apartado enfocado en la sincronización bidireccional mediante API para el monitoreo del progreso académico y calificaciones del Campus Virtual.

Certificaciones Digitales: Sección dedicada a la emisión validada de certificados mediante tecnología QR, gestión centralizada de microcredenciales y sistema de puntaje profesional.



Para el funcionamiento del portal estudiantil, se configuraron y estructuraron los contenidos estáticos principales dentro del sistema de administración de WordPress. Como se evidencia en el panel de control institucional, la arquitectura del sitio cuenta con las siguientes páginas esenciales:

Portal de Matrículas: Página central dedicada al flujo de inscripción académica e interacción estudiantil. Esta sección integra el shortcode y la lógica de backend para procesar altas de cursos directamente en la base de datos institucional.

Privacy Policy (Política de Privacidad): Página predeterminada del sistema orientada al cumplimiento de normativas de privacidad y seguridad de los datos de los usuarios.

Sample Page (Página de Ejemplo): Elemento de referencia inicial generado por el núcleo de WordPress durante su instalación base.

REQUERIMIENTOS

Define los recursos mínimos y recomendados necesarios a nivel de hardware y software para garantizar el correcto funcionamiento, estabilidad y rendimiento de la infraestructura del proyecto.

1 Requerimientos de Hardware

Debido a que el servidor aloja múltiples servicios concurrentes (servidor web, dos plataformas dinámicas como WordPress y Moodle, base de datos relacional, servidores de archivos y paneles de administración), se deben diferenciar los recursos mínimos para entornos de prueba o virtualización frente a los recomendados para entorno de producción o evaluación técnica.

Componente	Requerimiento Mínimo	Requerimiento Recomendado	Justificación Técnica
Procesador (CPU)	2 Cores / vCPUs (2.0 GHz)	4 Cores / vCPUs (2.5 GHz o superior)	Garantiza la capacidad de procesamiento multihilo para atender peticiones dinámicas de PHP y consultas concurrentes en MariaDB.
Memoria RAM	4 GB DDR4	8 GB DDR4 / DDR5	Necesaria para mantener en memoria los servicios concurrentes de Apache, MariaDB, Moodle y el almacenamiento en caché del sistema.
Almacenamiento	30 GB HDD / SSD	60 GB SSD NVMe	Espacio destinado para el SO (Ubuntu Server), repositorios web, datos de Moodle (moodledata), bases de datos y archivos de respaldo.

Componente	Requerimiento Mínimo	Requerimiento Recomendado	Justificación Técnica
Memoria SWAP	2 GB	4 GB	Actúa como área de intercambio en disco para prevenir fallos por agotamiento de memoria RAM física (<i>Out-Of-Memory Killer</i>).
Red (NIC)	1 Tarjeta Ethernet (100 Mbps)	1 Tarjeta Ethernet Gigabit (1 Gbps)	Requerida para el acceso a la red local/internet, gestión remota (SSH/Webmin) y transferencias de archivos vía FTP/SFTP.

2 Requerimientos de Software

Los requerimientos de software contemplan los sistemas operativos, hipervisores, herramientas de desarrollo y paquetes de aplicaciones necesarios tanto en el lado del servidor como en el equipo cliente del administrador/evaluador.

2.1 Software en el Servidor (Server-Side)

Sistema Operativo: Ubuntu Server 24.04 LTS (o versión 22.04 LTS) de 64 bits (x86_64).

Intérprete de Comandos: GNU Bash versión 5.x o superior.

Servidor Web: Apache HTTP Server (apache2) v2.4.x.

Base de Datos: MariaDB Server (mariadb-server) v10.11.x o superior.

Entorno de Procesamiento: PHP v8.2 o v8.3 con sus módulos esenciales:

- php-mysql, php-xml, php-gd, php-mbstring, php-curl, php-zip, php-intl, php-soap.

Servicios de Archivos y Seguridad:

- vsftpd (Servidor FTP).
- openssh-server (Servicio SSH/SFTP).
- ufw (Uncomplicated Firewall).



Gestión y Monitoreo:

- webmin (Panel de administración web).
- cron / crontab (Planificador de tareas).
- Utilitarios de sistema: curl, wget, tar, gzip, zip, unzip, htop, net-tools.

Aplicaciones Web (CMS / LMS):

- WordPress (última versión estable).
- Moodle LMS (última versión estable compatible con la versión de PHP).

2.2 Software en el Cliente / Administrador (Client-Side)

Sistema Operativo: Windows 10/11, Linux (Ubuntu/Debian/OpenSUSE) o macOS.

Hipervisor / Virtualización (si aplica): Oracle VirtualBox, VMware Workstation o KVM.

Navegador Web: Google Chrome, Mozilla Firefox o Microsoft Edge para acceso a los sitios web (WordPress, Moodle) e interfaz de Webmin.

Cliente SSH y SFTP:

- Terminal nativa (Linux/macOS) o PowerShell / CMD / PuTTY (Windows).
- FileZilla o WinSCP para transferencias de archivos vía FTP/SFTP.

Editor de Texto: Visual Studio Code, Notepad++ o Nano/Vim para la edición de scripts en Bash y archivos de configuración .conf o .sh.

Estructura de Carpetas y Scripts

Para mantener la modularidad y el orden en el sistema, todos los scripts de automatización se ubican en una carpeta centralizada dentro del directorio raíz del usuario administrador (por ejemplo, /root/scripts/ o /home/admin/scripts/).

```
/root/scripts/  
├── 01-update.sh  
├── 02-lamp.sh  
├── 03-ftp.sh  
└── 04-sftp.sh
```




- 05-wordpress.sh
- 06-moodle.sh
- 07-vhosts.sh
- 08-security.sh
- 09-backup.sh
- 10-webmin.sh
- 11-monitor.sh

Estructura de Directorios del Servidor Web:

- /var/www/html/wordpress/: Directorio raíz de la aplicación WordPress.
- /var/www/html/moodle/: Directorio raíz de la plataforma Moodle.
- /var/moodledata/: Directorio de datos seguro para Moodle (ubicado fuera de la raíz web por seguridad).
- /etc/apache2/sites-available/: Directorio que contiene los archivos de configuración .conf de los Virtual Hosts.
- /var/backups/proyecto/: Directorio destino para los respaldos automáticos de bases de datos y archivos del sistema.



DESCRIPCIÓN DE SCRIPTS

A continuación, se detalla la función específica, los comandos clave y la responsabilidad de cada uno de los 11 scripts que componen la solución:

01-update.sh

- **Propósito:** Actualizar el índice de paquetes y el sistema operativo a su última versión estable.
- **Operaciones principales:** Ejecución de apt update, apt upgrade -y y limpieza de paquetes obsoletos con apt autoremove -y.

02-lamp.sh

- **Propósito:** Instalar y configurar el Stack LAMP (Linux, Apache, MariaDB, PHP).
- **Operaciones principales:** Instalación de apache2, mariadb-server, php y sus librerías requeridas (php-mysql, php-gd, php-xml, etc.). Habilitación e inicio de los servicios vía systemctl.

03-ftp.sh

- **Propósito:** Implementar y configurar el servicio de transferencia de archivos FTP.
- **Operaciones principales:** Instalación del demonio vsftpd, edición del archivo /etc/vsftpd.conf, creación de usuarios de transferencia y habilitación de permisos de escritura.

04-sftp.sh

- **Propósito:** Configurar la transferencia segura de archivos mediante SSH File Transfer Protocol.
- **Operaciones principales:** Creación de grupos de usuarios dedicados, configuración de encarcelamiento (*Chroot Jail*) en /etc/ssh/sshd_config y asignación de permisos sobre las carpetas del servidor web.

05-wordpress.sh

- **Propósito:** Automatizar la descarga, instalación y configuración inicial de WordPress.
- **Operaciones principales:** Creación de la base de datos y usuario en MariaDB, descarga del paquete comprimido de WordPress, extracción en /var/www/html/wordpress, configuración de wp-config.php y ajuste de permisos (chown -R www-data:www-data).

06-moodle.sh

- **Propósito:** Desplegar la plataforma educativa Moodle y sus dependencias.



- **Operaciones principales:** Creación de la base de datos MariaDB (con cotejo utf8mb4_unicode_ci), creación del directorio /var/moodledata fuera de la raíz web, descarga del código fuente de Moodle y ajuste de permisos de seguridad.

07-vhosts.sh

- **Propósito:** Configurar los anfitriones virtuales en el servidor web Apache.
- **Operaciones principales:** Creación automática de los archivos .conf en /etc/apache2/sites-available/ para asociar dominios/subdominios a WordPress y Moodle, activación de sitios con a2ensite y recarga de Apache.

08-security.sh

- **Propósito:** Implementar políticas de seguridad en el cortafuegos UFW y endurecimiento (*hardening*) de SSH.
- **Operaciones principales:** Habilitación de UFW, apertura exclusiva de puertos permitidos (80, 443, 22, 21, 10000), y deshabilitación del acceso root directo en SSH.

09-backup.sh

- **Propósito:** Automatizar las copias de seguridad de las bases de datos y los sitios web.
- **Operaciones principales:** Exportación de bases de datos mediante mysqldump, compresión de carpetas web con tar -czvf y almacenamiento programado en /var/backups/.

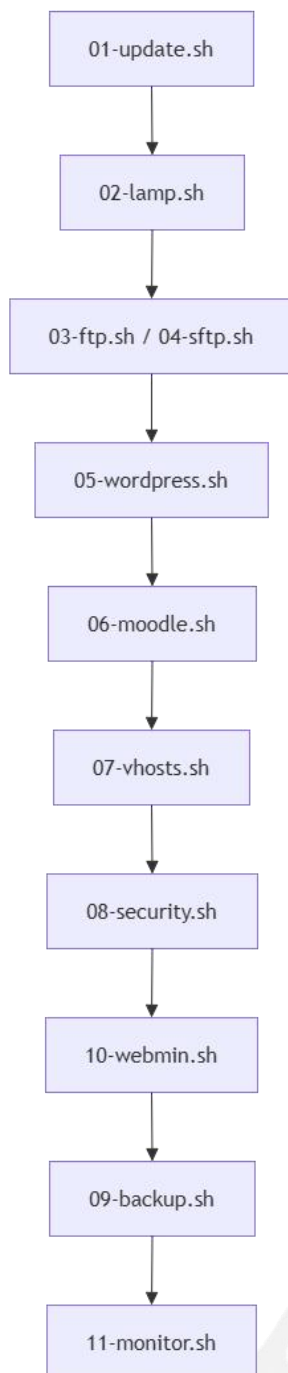
10-webmin.sh

- **Propósito:** Instalar el panel de administración gráfica en entorno web.
- **Operaciones principales:** Adición del repositorio oficial de Webmin, importación de claves GPG, instalación del paquete y apertura del puerto 10000/TCP en UFW.

11-monitor.sh

- **Propósito:** Inspeccionar y reportar el estado de salud y consumo de recursos del servidor.
- **Operaciones principales:** Verificación del estado de servicios activos (systemctl is-active), monitoreo de uso de CPU, memoria RAM, espacio en disco y lectura de registros de errores (*logs*).

FLUJO DE EJECUCIÓN





Explicación de la Secuencia Operativa:

1. **Base del Sistema (01-update.sh):** Se actualiza el sistema para asegurar que todos los paquetes a descargar sean las versiones más recientes y seguras.
2. **Servicios Core (02-lamp.sh):** Se instala la base de servidor web y datos, necesaria para recibir las aplicaciones web posteriores.
3. **Servicios de Archivos (03-ftp.sh y 04-sftp.sh):** Se preparan las vías de acceso a archivos antes de descargar los CMS.
4. **Despliegue de Aplicaciones (05-wordpress.sh y 06-moodle.sh):** Se instalan las plataformas que dependen directamente del Stack LAMP previo.
5. **Enrutamiento Web (07-vhosts.sh):** Una vez que las carpetas de las aplicaciones existen, se mapean los nombres de dominio mediante los Virtual Hosts de Apache.
6. **Seguridad y Administración (08-security.sh y 10-webmin.sh):** Se asegura el servidor cerrando puertos no utilizados y se habilita la interfaz gráfica de administración.
7. **Mantenimiento y Monitoreo (09-backup.sh y 11-monitor.sh):** Se automatizan las tareas de respaldo en segundo plano y se ejecuta la verificación del estado general del servidor.

Descripción de los Scripts Utilizados

Script 0: Configuración de IP Estática (Netplan)

Qué hace en general

Configura una dirección IP estática en un servidor Ubuntu, reemplazando la asignación por DHCP, y se asegura de que esa configuración persista automáticamente tras cada reinicio del servidor.

Cómo funciona paso a paso

- **Verificación de permisos:** Comprueba que se ejecute como root; si no, se detiene.
- **Definición de variables:** Declara interfaz, IP, prefijo, gateway y DNS al inicio del script.
- **Respaldo del archivo de configuración:** Copia 50-cloud-init.yaml a .bak antes de modificarlo.
- **Generación de la nueva configuración:** Sobrescribe el YAML de Netplan con la IP fija, ruta por defecto y DNS.
- **Aplicación de la configuración:** Ejecuta Netplan generate y netplan apply.
- **Confirmación visual:** Muestra un resumen y ejecuta ip addr show para verificar.
- **Persistencia tras reinicio (Crontab):** Crea una entrada @reboot en /etc/cron.d/ que vuelve a ejecutar el script en cada arranque.

Requisitos y dependencias asumidas

Ubuntu Server con Netplan, interfaz ens18, archivo de configuración existente, ejecución como root, servicio cron activo.

Resultado final

IP estática aplicada al instante, con respaldo de la configuración anterior y persistencia garantizada mediante cron ante cualquier reinicio.



Bloques de código más importantes

```
cat > $ARCHIVO << EOF
```

```
network:
```

```
  version: 2
```

```
  ethernet:
```

```
    $INTERFAZ:
```

```
      dhcp4: false
```

```
      addresses:
```

```
        - $IP/$PREFIJO
```

```
      routes:
```

```
        - to: default
```

```
        via: $GATEWAY
```

```
      nameservers:
```

```
        addresses:
```

```
          - $DNS1
```

```
          - $DNS2
```

```
EOF
```

Este bloque es el corazón del script: usa un heredoc (<< EOF ... EOF) para generar dinámicamente el archivo YAML de Netplan, sustituyendo las variables definidas antes. dhcp4:

false desactiva la asignación automática, y el resto define la IP fija, la ruta por defecto y los servidores DNS. Al ser generado con variables, el mismo bloque sirve para cualquier IP sin tener que reescribir la sintaxis YAML manualmente.

```
SCRIPT_PATH=$(realpath "$0")

CRON_FILE="/etc/cron.d/static_ip_on_reboot"

cat > $CRON_FILE << EOF

@reboot root /bin/bash $SCRIPT_PATH > /var/log/static_ip_reboot.log 2>&1

EOF
```

Este bloque es el que le da persistencia a la configuración. `realpath "$0"` obtiene la ruta absoluta del propio script (sin importar desde dónde se llamó), y la tarea `@reboot` de cron hace que ese mismo script se vuelva a ejecutar automáticamente cada vez que el servidor arranca, redirigiendo toda la salida a un log para poder auditar qué pasó en cada reinicio.

Script 1: 01-update.sh

Qué hace en general

Actualiza el sistema operativo e instala un conjunto de utilidades base necesarias para que los scripts siguientes del proyecto UNAH-CONECTA funcionen correctamente.

Cómo funciona paso a paso

- **Modo estricto:** Activa `set -euo pipefail` para detenerse ante cualquier error.
- **Resolución de rutas:** Calcula `SCRIPT_DIR` y `BASE_DIR` de forma relativa a su propia ubicación.

- **Carga de dependencias compartidas:** Importa helpers.sh (funciones de salida como paso, ok, info, error) y config.env.
- **Verificación de privilegios:** Llama a require_root.
- **Modo no interactivo de apt:** Exporta DEBIAN_FRONTEND=noninteractive.
- **Paso 1 — Actualización de índices:** Ejecuta apt-get update.
- **Paso 2 — Actualización de paquetes:** Verifica cuántos paquetes tienen actualización pendiente y aplica apt-get upgrade solo si hace falta.
- **Paso 3 — Instalación idempotente de utilidades base:** Revisa con dpkg -s cuáles paquetes de una lista predefinida faltan, e instala solo esos.
- **Paso 4 — Comprobación de reinicio requerido:** Verifica /var/run/reboot-required y advierte si existe.

Requisitos y dependencias asumidas

Debian/Ubuntu, root/sudo, helpers.sh y config.env presentes, conexión a internet.

Resultado final

Sistema actualizado con todas las utilidades base necesarias instaladas, listo para los scripts posteriores.

Bloques de código más importantes

```
REQUIRED_PACKAGES=(curl wget unzip software-properties-common ca-certificates gnupg lsb-release ufw)

MISSING_PACKAGES=()

for pkg in "${REQUIRED_PACKAGES[@]"; do

    if ! dpkg -s "$pkg" >/dev/null 2>&1; then

        MISSING_PACKAGES+=("$pkg")
```



```
fi

done

if [ $#MISSING_PACKAGES[@] -eq 0 ]; then

    ok "Todas las utilidades base ya están instaladas."

else

    apt-get install -y -qq "${MISSING_PACKAGES[@]}" || error "..."

fi
```

Este es el bloque más importante del script porque implementa la idempotencia: en vez de simplemente instalar todo cada vez, recorre la lista de paquetes requeridos, consulta con `dpkg -s` cuál falta realmente, y arma un arreglo (`MISSING_PACKAGES`) solo con lo que hace falta instalar. Esto permite ejecutar el script las veces que sea necesario sin efectos negativos.

```
UPGRADABLE_LIST=$(apt list --upgradable 2>/dev/null | tail -n +2 || true)

UPGRADABLE_COUNT=$(echo "$UPGRADABLE_LIST" | grep -v '^$' | wc -l || echo 0)

if [ "$UPGRADABLE_COUNT" -eq 0 ]; then

    ok "El sistema operativo ya se encuentra actualizado (0 paquetes pendientes)."
```

```
else
```



```
apt-get upgrade -y -qq >/dev/null 2>&1 || error "..."
```

fi

El script no aplica apt-get upgrade a ciegas: primero cuenta cuántos paquetes tienen actualización pendiente (tail -n +2 descarta la línea de encabezado, y grep -v '^\$' filtra líneas vacías para un conteo preciso). Solo si ese número es mayor a cero se ejecuta la actualización real, evitando trabajo innecesario.

Script 2: 02-apache.sh

Qué hace en general

Instala Apache2, habilita los módulos necesarios para trabajar como proxy hacia PHP-FPM, y deja el servicio corriendo y verificado.

Cómo funciona paso a paso

- **Configuración inicial:** Igual que el script 1 (modo estricto, rutas, helpers, root, apt no interactivo).
- **Paso 1 — Instalación de Apache2:** Comprueba si ya existe con `command -v apache2`; si no, actualiza índices e instala el paquete.
- **Paso 2 — Habilitar módulos:** Activa `rewrite`, `proxy`, `proxy_fcgi` y `setenvif` con `a2enmod`.
- **Paso 3 — Respaldo del sitio por defecto:** Copia `000-default.conf` a `.bak`, sin sobrescribir un respaldo ya existente.
- **Paso 4 — Validar sintaxis y reiniciar:** Ejecuta `apache2ctl configtest`; si es válido, habilita y reinicia el servicio.
- **Paso 5 — Verificación del estado:** Confirma con `systemctl is-active` que Apache quedó corriendo.

Requisitos y dependencias asumidas

Debian/Ubuntu con `systemd`, `root/sudo`, `helpers.sh` y `config.env`, ejecución posterior a `01-update.sh`.

Resultado final

Apache2 instalado, habilitado al arranque, con los módulos necesarios para PHP-FPM activos, respaldo del sitio por defecto guardado, y servicio verificado como activo.

Bloques de código más importantes

```
a2enmod rewrite proxy proxy_fcgi setenvif >/dev/null 2>&1 || error "..."
```

Aunque es una sola línea, es el bloque más determinante del script a nivel funcional: define la arquitectura del servidor web. `rewrite` habilita las URLs amigables que WordPress necesita para sus permalinks; `proxy` y `proxy_fcgi` permiten que Apache reenvíe las peticiones PHP a un proceso FastCGI externo (PHP-FPM, en vez del clásico `mod_php`); `setenvif` permite condicionar variables de entorno según cabeceras HTTP.

```
if apache2ctl configtest >/dev/null 2>&1; then  
  
    ok "Sintaxis de configuración de Apache2 válida."  
  
else  
  
    error "La prueba de sintaxis de Apache2 (configtest) ha fallado. Abortando reinicio."  
  
fi  
  
systemctl restart apache2 >/dev/null 2>&1 || error "..."
```

Este bloque es una salvaguarda importante: antes de reiniciar Apache, valida que la configuración actual no tenga errores de sintaxis. Si el `configtest` fallara y el script igual reiniciara el servicio, Apache podría quedar caído por completo, a diferencia de un reload. Validar antes de reiniciar evita dejar el servidor web fuera de servicio.

Script 3: 03-mariadb.sh

Qué hace en general

Instala MariaDB, aplica una securización inicial no interactiva equivalente a `mysql_secure_installation`, y crea las bases de datos y usuarios necesarios para WordPress y Moodle, validando al final que las credenciales funcionen.

Cómo funciona paso a paso

- **Configuración inicial:** Modo estricto, rutas, helpers, root.
- **Validación de variables sensibles:** Verifica que `DB_WP_PASS` y `DB_MOODLE_PASS` existan en `config.env` antes de continuar.
- **Ajustes para instalación silenciosa:** `NEEDRESTART_MODE=a`, `NEEDRESTART_SUSPEND=1` y opciones extra de `apt` para evitar prompts.
- **Paso 1 — Instalación de MariaDB:** Instala `mariadb-server` y `mariadb-client` si no existen.
- **Paso 2 — Habilitar y arrancar el servicio:** `systemctl enable` y `start`, con verificación de estado activo.
- **Paso 3 — Securitización inicial:** Elimina usuarios anónimos, bloquea root remoto, borra la base test.
- **Paso 4 — Base de datos y usuario para WordPress:** Crea base y usuario si no existen, y otorga privilegios siempre.
- **Paso 5 — Base de datos y usuario para Moodle:** Repite la misma lógica para Moodle.
- **Paso 6 — Verificación de conexiones:** Prueba login real con cada usuario creado contra su base de datos.

Requisitos y dependencias asumidas

Debian/Ubuntu con `systemd`, `root/sudo`, `helpers.sh` y `config.env` con las variables de nombres, usuarios y contraseñas de ambas bases, acceso root de MariaDB por socket Unix (sin contraseña).

Resultado final

MariaDB instalado, activo, securizado, con dos bases de datos independientes (WordPress y Moodle), cada una con su usuario dedicado, privilegios correctos y conexión verificada.

Bloques de código más importantes

```
mysql -s -e "  
  
DELETE FROM mysql.user WHERE User=";  
  
DELETE FROM mysql.user WHERE User='root' AND Host NOT IN ('localhost', '127.0.0.1', '::1');  
  
DROP DATABASE IF EXISTS test;  
  
DELETE FROM mysql.db WHERE Db='test' OR Db='test\\_%';  
  
FLUSH PRIVILEGES;  
  
" || error "..."
```

Este bloque reemplaza al asistente interactivo `mysql_secure_installation`, automatizando sus pasos clave mediante SQL directo: elimina usuarios anónimos, restringe el usuario root para que solo pueda conectarse localmente, y elimina la base de datos test junto con sus referencias en la tabla de privilegios. `FLUSH PRIVILEGES` recarga la tabla de permisos en memoria.

```
DB_WP_EXISTS=$(mysql -sN -e "SHOW DATABASES LIKE '${DB_WP_NAME}';")  
  
if [[ "$DB_WP_EXISTS" == "$DB_WP_NAME" ]]; then  
  
    advertencia "La base de datos de WordPress ya existe."  
  
else
```




```
mysql -e "CREATE DATABASE \`${DB_WP_NAME}\` CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;" ||  
error "..."  
  
fi
```

Este patrón (repetido también para Moodle) es clave para la idempotencia del script a nivel de base de datos: usa SHOW DATABASES LIKE con las flags -sN (salida silenciosa) para comparar directamente. Solo crea la base si no existe. El uso de utf8mb4/utf8mb4_unicode_ci es la codificación recomendada oficialmente por WordPress y Moodle.

```
if mysql -u "${DB_WP_USER}" -p"${DB_WP_PASS}" -h "localhost" "${DB_WP_NAME}" -e "SELECT 1;" >/dev/null 2>&1;  
then  
  
    ok "Conexión de prueba exitosa para el usuario de WordPress."  
  
else  
  
    error "Fallo de autenticación para el usuario de WordPress."  
  
fi
```

Este bloque de verificación final es importante porque no se conforma con que los comandos CREATE USER y GRANT se hayan ejecutado sin errores de sintaxis: intenta una conexión real con las credenciales creadas y ejecuta una consulta trivial (SELECT 1), detectando fallos de autenticación de forma inmediata.

Script 4: 04-php.sh

Qué hace en general

Instala PHP 8.3 (versión nativa de los repositorios de Ubuntu Server 24.04 LTS, sin usar Docker) junto con todas las extensiones necesarias para que WordPress y Moodle 4.5 LTS funcionen correctamente, configura el pool de PHP-FPM con parámetros de rendimiento adecuados, y deja el servicio corriendo y verificado, incluyendo la existencia física del socket Unix que Apache usará para comunicarse con PHP.

Cómo funciona paso a paso

- **Configuración inicial:** Modo estricto, resolución de rutas, carga de helpers.sh y config.env, verificación de root. Define `PHP_VERSION="8.3"`.
- **Paso 1 — Verificación de PHP base:** Comprueba si ya existe el comando php y obtiene su versión exacta mediante `php -r`.
- **Paso 2 — Instalación de PHP-FPM y extensiones:** Arma dinámicamente los nombres de paquete (`php8.3-<extensión>`) e instala solo los faltantes.
- **Paso 3 — Verificación de módulos críticos:** Confirma con `php -m` que las extensiones críticas estén activas en tiempo de ejecución.
- **Paso 4 — Configuración del pool de PHP-FPM:** Ajusta `pm.max_children`, `memory_limit` y `upload_max_filesize` en `www.conf`.
- **Paso 5 — Habilitación, reinicio y verificación del servicio:** Habilita/reinicia `php8.3-fpm` y verifica la existencia del socket Unix.

Requisitos y dependencias asumidas

Ubuntu Server 24.04 LTS, root/sudo, helpers.sh y config.env, ejecución posterior a 02-apache.sh.

Resultado final

PHP 8.3 y todas sus extensiones necesarias instaladas y verificadas, con el pool de PHP-FPM ajustado, el servicio activo y habilitado al arranque, y el socket Unix disponible para que Apache pueda enviarle las peticiones PHP.



Bloques de código más importantes

```
VERSION_ACTUAL=$(php -r 'echo PHP_MAJOR_VERSION.".".PHP_MINOR_VERSION;')

if [[ "$VERSION_ACTUAL" == "$PHP_VERSION" ]]; then

    advertencia "PHP ${PHP_VERSION} ya está instalado y activo en el sistema."

    PHP_INSTALADO=true

fi
```

Este bloque ejecuta código PHP en línea (php -r) para leer las constantes internas de versión y compararlas con la requerida. Esto evita que el sistema tenga instalada, por ejemplo, PHP 8.1 de una instalación previa y el script asuma erróneamente que la versión correcta ya está lista.

```
EXTENSIONS=("fpm" "cli" "mysql" "curl" "gd" "mbstring" "xml" "zip" "soap" "intl" "bcmath" "xmlrpc")

PAQUETES_A_INSTALAR=()

for ext in "${EXTENSIONS[@]};" do

    PKG_NAME="php${PHP_VERSION}-${ext}"

    if ! dpkg -s "$PKG_NAME" >/dev/null 2>&1; then

        PAQUETES_A_INSTALAR+=("$PKG_NAME")

    fi

done
```

Construye dinámicamente los nombres de paquete concatenando la versión de PHP con cada extensión, lo que hace que el script sea fácilmente adaptable a otra versión de PHP con solo

cambiar la variable PHP_VERSION. Aplica el mismo patrón de idempotencia visto en scripts anteriores.

```
if grep -q "^pm.max_children = 15" "$FPM_POOL_CONF"; then

    info "pm.max_children ya está configurado en 15."

else

    sed -i 's/^\?pm.max_children = */pm.max_children = 15/' "$FPM_POOL_CONF"

fi
```

Este patrón (repetido para memory_limit y upload_max_filesize) edita un archivo de configuración existente de manera segura e idempotente: comprueba si el valor deseado ya está aplicado y, si no, usa sed con una expresión regular que reemplaza la línea exista comentada o no, evitando duplicar líneas tras múltiples ejecuciones.

```
SOCKET_PATH="/run/php/php${PHP_VERSION}-fpm.sock"

if [ -S "$SOCKET_PATH" ]; then

    ok "Socket de conexión localizado correctamente."

else

    error "No se encontró el socket de PHP-FPM en la ruta esperada."

fi
```

Esta verificación final valida el punto exacto de integración entre Apache y PHP-FPM: el operador -S comprueba específicamente que el archivo exista y sea un socket Unix, cerrando el ciclo de verificación real del stack.

Script 5: 05-wordpress.sh

Qué hace en general

Descarga, configura e instala WordPress de forma completamente desatendida utilizando WP-CLI, la herramienta oficial de línea de comandos de WordPress. Al finalizar, deja el sitio funcional con su base de datos inicializada y los permisos de archivos asegurados.

Cómo funciona paso a paso

- **Configuración inicial y validación:** Modo estricto, rutas, helpers, root, apt no interactivo, y validación de que existan en config.env las variables WP_TITLE, WP_ADMIN_USER, WP_ADMIN_PASS y WP_ADMIN_EMAIL.
- **Paso 1 — Instalación idempotente de WP-CLI:** Si el comando wp no existe, descarga el archivo .phar oficial desde el repositorio de GitHub de WP-CLI, valida que no esté vacío/corrupto, le da permisos de ejecución y lo mueve a /usr/local/bin/wp. Luego valida su funcionamiento con wp --info.
- **Paso 2 — Verificación de idempotencia de la instalación:** Si ya existe wp-config.php en el directorio destino, asume que WordPress ya está instalado y termina el script con exit 0, sin tocar nada.
- **Paso 3 — Preparación del directorio destino:** Crea el directorio si no existe; si existe y no está vacío, verifica si corresponde a una instalación parcial (detecta wp-load.php) o aborta para no sobrescribir archivos desconocidos.
- **Paso 4 — Descarga del núcleo de WordPress:** Usa wp core download con --locale=es_ES para obtener WordPress en español, y verifica que wp-load.php se haya generado.
- **Paso 5 — Generación de wp-config.php:** Usa wp config create con las credenciales de base de datos (nombre, usuario, contraseña, host), y valida la conexión real a la base con wp db check.

- **Paso 6 — Instalación desatendida del sitio:** Ejecuta wp core install con el dominio, título, usuario/contraseña/correo del administrador, completando la instalación sin intervención manual.
- **Paso 7 — Permisos y propiedad (seguridad):** Cambia el propietario de todos los archivos a www-data (el usuario de Apache), aplica 755 a directorios y 644 a archivos, y restringe wp-config.php a 600 por contener credenciales sensibles.
- **Paso 8 — Verificación final:** Confirma con wp core is-installed que la instalación quedó marcada como completa, y hace una petición HTTP local simulando la cabecera Host del dominio para comprobar la respuesta del servidor.

Requisitos y dependencias asumidas

Debian/Ubuntu, root/sudo, helpers.sh y config.env con DOMAIN_WP, PATH_WP, DB_WP_NAME, DB_WP_USER, DB_WP_PASS, además de las nuevas variables WP_TITLE, WP_ADMIN_USER, WP_ADMIN_PASS y WP_ADMIN_EMAIL. Debe ejecutarse después de 03-mariadb.sh (la base de datos debe existir) y 04-php.sh (PHP debe estar disponible para que WP-CLI funcione).

Resultado final

WordPress descargado en español, instalado y conectado a su base de datos, con un usuario administrador ya creado, permisos de archivos asegurados (credenciales protegidas con permisos restrictivos), y una verificación tanto a nivel de WP-CLI como de respuesta HTTP real.

Bloques de código más importantes

```
if [ -f "${PATH_WP}/wp-config.php" ]; then  
  
    advertencia "WordPress ya parece estar instalado. Omitiendo el despliegue."  
  
    exit 0
```



fi

Este es el control de idempotencia más importante del script: en vez de solo advertir y continuar (como en scripts anteriores), aquí directamente termina la ejecución con exit 0 si detecta una instalación previa. Esto evita cualquier riesgo de que un wp core install posterior sobrescriba o dañe una instalación de WordPress que ya está en producción.

```
wp core install \

--url="${DOMAIN_WP}" \

--title="${WP_TITLE}" \

--admin_user="${WP_ADMIN_USER}" \

--admin_password="${WP_ADMIN_PASS}" \

--admin_email="${WP_ADMIN_EMAIL}" \

--path="${PATH_WP}" \

--allow-root >/dev/null 2>&1 || error "..."
```

Este es el comando central del script: reemplaza por completo el típico asistente web de instalación de WordPress (los 5 minutos famosos de configuración por navegador) con un solo comando de línea. El flag --allow-root es necesario porque WP-CLI, por diseño, bloquea su ejecución como root a menos que se indique explícitamente, ya que ejecutar WordPress como root normalmente no es una buena práctica de seguridad.

```
chown -R www-data:www-data "$PATH_WP" || error "..."  
  
find "$PATH_WP" -type d -exec chmod 755 {} + || error "..."  
  
find "$PATH_WP" -type f -exec chmod 644 {} + || error "..."  
  
chmod 600 "${PATH_WP}/wp-config.php" || error "..."
```

Este bloque aplica el principio de mínimo privilegio a nivel de sistema de archivos: 755 en directorios permite que Apache (www-data) los recorra y liste; 644 en archivos permite lectura pero no ejecución directa de scripts arbitrarios; y 600 en wp-config.php (que contiene las credenciales de la base de datos en texto plano) hace que solo el propietario del archivo pueda leerlo, bloqueando el acceso de cualquier otro usuario del sistema.

Script 6: 06-moodle.sh

Qué hace en general

Descarga el código fuente de Moodle 4.5 LTS directamente desde su repositorio oficial de Git, prepara el directorio de datos fuera de la raíz web (por seguridad), ejecuta el instalador CLI oficial de Moodle de forma no interactiva, asegura los permisos de archivos, y configura la tarea cron obligatoria que Moodle necesita para su funcionamiento continuo (envío de notificaciones, tareas programadas, mantenimiento, etc.).

Cómo funciona paso a paso

- **Configuración inicial y validación:** Modo estricto, rutas, helpers, root, y validación de que existan en config.env: MOODLE_DATA_DIR, MOODLE_BRANCH, MOODLE_SITE_NAME, MOODLE_ADMIN_USER, MOODLE_ADMIN_PASS y MOODLE_ADMIN_EMAIL.
- **Paso 1 — Verificación de idempotencia:** Si ya existe config.php en el directorio de Moodle, se asume instalación previa y el script termina con exit 0.

- **Paso 2 — Descarga del código fuente:** Verifica que git esté instalado y que el directorio destino esté vacío, y clona el repositorio oficial de Moodle en GitHub usando la rama especificada (--branch) con --depth 1 (clonación superficial, más rápida). Luego valida la existencia de version.php y que contenga la cadena "4.5".
- **Paso 3 — Preparación del directorio de datos (moodledata):** Valida con realpath -m que MOODLE_DATA_DIR no sea un subdirectorio del webroot de Moodle (requisito de seguridad de la propia plataforma), lo crea si no existe, y le asigna propietario www-data con permisos restrictivos 770.
- **Paso 4 — Instalación no interactiva:** Cambia temporalmente la propiedad del webroot a www-data, y ejecuta el instalador oficial admin/cli/install.php como el usuario www-data (con sudo -u), pasando todos los parámetros de base de datos, dominio, directorio de datos y datos del administrador. La salida se redirige a un archivo de log temporal para diagnóstico.
- **Paso 5 — Permisos finales:** Vuelve a aplicar propiedad www-data y permisos 755/644 de forma recursiva sobre todo el directorio, y restringe config.php a 600.
- **Paso 6 — Configuración del cron de Moodle:** Verifica si ya existe la entrada en el crontab del usuario www-data; si no, la agrega para que se ejecute cada minuto (* * * * *).
- **Paso 7 — Verificación final:** Confirma que config.php contiene el nombre correcto de la base de datos, y que la entrada de cron quedó efectivamente registrada.

Requisitos y dependencias asumidas

Debian/Ubuntu, root/sudo, git instalado, helpers.sh y config.env con PATH_MOODLE, DOMAIN_MOODLE, DB_MOODLE_NAME, DB_MOODLE_USER, DB_MOODLE_PASS y las nuevas variables de Moodle. Debe ejecutarse después de 03-mariadb.sh y 04-php.sh.

Resultado final

Moodle 4.5 LTS instalado desde el código fuente oficial, con su directorio de datos protegido y ubicado fuera de la raíz web, base de datos conectada, administrador creado, permisos de archivos asegurados, y la tarea cron obligatoria de Moodle funcionando cada minuto.



Bloques de código más importantes

```
REAL_PATH_MOODLE=$(realpath -m "$PATH_MOODLE")

REAL_MOODLE_DATA=$(realpath -m "$MOODLE_DATA_DIR")

if [[ "$REAL_MOODLE_DATA" == "$REAL_PATH_MOODLE"* ]]; then

    error "MOODLE_DATA_DIR no puede ser un subdirectorio de PATH_MOODLE."

fi
```

Esta validación de seguridad es un requisito explícito de la propia documentación oficial de Moodle: el directorio moodledata contiene archivos subidos por los usuarios y datos de sesión, y si estuviera dentro de la carpeta servida por Apache, cualquier persona podría acceder a esos archivos directamente por HTTP sin pasar por los controles de permisos de Moodle. El script usa `realpath -m` para resolver ambas rutas a su forma absoluta y comparar con un simple prefijo de cadena, evitando que se apruebe una configuración insegura.

```
sudo -u www-data php \

    -d max_input_vars=5000 \

    -d memory_limit=256M \

    -d max_execution_time=0 \

    "${PATH_MOODLE}/admin/cli/install.php" \

    --non-interactive \
```

```
--agree-license \
```

```
...
```

Este es el comando central del script. Ejecuta el instalador como el usuario www-data (no como root) usando sudo -u, porque Moodle necesita crear config.php con el propietario correcto desde el inicio. Los tres flags -d sobrescriben valores de php.ini solo para esta ejecución puntual: max_input_vars=5000 es crítico porque el formulario de instalación de Moodle envía muchos más campos que el límite por defecto de PHP (1000), lo que causaría fallos silenciosos y difíciles de diagnosticar si no se ajustara.

```
if crontab -u www-data -l 2>/dev/null | grep -q "${PATH_MOODLE}/admin/cli/cron.php"; then

    info "El cron de Moodle ya está configurado."

else

    (crontab -u www-data -l 2>/dev/null || true; echo "* * * * * /usr/bin/php ${PATH_MOODLE}/admin/cli/cron.php >/dev/null
    2>&1") | crontab -u www-data -

fi
```

Este bloque agrega la tarea cron de forma idempotente y sin destruir tareas cron que ya existieran para www-data. En vez de sobrescribir directamente el crontab, primero lee el contenido actual (o una cadena vacía si no existe ninguno) y le concatena la nueva línea, para luego volver a cargar todo junto con crontab -. El cron de Moodle cada minuto es un requisito de la plataforma, ya que de él dependen tareas como el envío de correos, la sincronización de cursos y el mantenimiento programado.

Script 7: 07-vhosts.sh

Qué hace en general

Genera y despliega de forma dinámica los Virtual Hosts de Apache para WordPress y Moodle, conectándolos a PHP-FPM mediante proxy_fcgi sobre socket Unix. Soporta dos escenarios: un modo temporal por IP con puertos distintos (por ejemplo, WordPress en el puerto 80 y Moodle en el 8080, útil mientras no hay dominios reales asignados) y un modo definitivo por dominio, donde ambos sitios responden en el puerto 80 diferenciados por ServerName.

Cómo funciona paso a paso

- **Configuración inicial y validación de dependencias:** Modo estricto, rutas, helpers, root. Valida que PHP_VERSION y SERVER_ADMIN_EMAIL existan en config.env, y confirma que el socket de PHP-FPM (generado en 04-php.sh) exista físicamente antes de continuar.
- **Paso 2 — Detección del modo de operación:** Define dos funciones auxiliares (extraer_host y extraer_puerto) que interpretan si DOMAIN_WP/DOMAIN_MOODLE vienen en formato "IP:puerto" o solo "dominio", separando el host del puerto (asumiendo 80 por defecto si no se especifica).
- **Paso 3 — Habilitar puerto extra en Apache:** Si Moodle usa un puerto distinto a 80, agrega una directiva Listen correspondiente en /etc/apache2/ports.conf, evitando duplicados.
- **Paso 4 — Función generadora de Virtual Hosts:** Define una función reutilizable (generar_vhost) que recibe host, puerto, ruta, nombre de archivo y tipo de AllowOverride, y genera el bloque <VirtualHost> completo con la conexión a PHP-FPM vía proxy:unix.
- **Paso 5 — Generación de configuraciones:** Llama a la función generadora dos veces: una para WordPress (con AllowOverride All, necesario para los permalinks vía .htaccess) y otra para Moodle (con AllowOverride None, ya que Moodle no usa .htaccess y esto mejora el rendimiento y la seguridad).
- **Paso 6 — Despliegue hacia Apache:** Copia los archivos generados a /etc/apache2/sites-available/, respaldando con timestamp cualquier configuración previa, y habilita cada sitio con a2ensite si no estaba ya habilitado.
- **Paso 7 — Desactivación del sitio por defecto:** Deshabilita 000-default.conf con a2dissite, para que no interfiera con los nuevos Virtual Hosts.

- **Paso 8 — Validación de sintaxis:** Ejecuta `apache2ctl configtest` antes de aplicar cualquier cambio real al servicio.
- **Paso 9 — Recarga y verificación real:** Recarga Apache con `systemctl reload`, confirma que el servicio sigue activo, y hace peticiones HTTP reales (con `curl`) a cada sitio para verificar el código de respuesta.

Requisitos y dependencias asumidas

Debian/Ubuntu con Apache2 y systemd, root/sudo, helpers.sh y config.env con DOMAIN_WP, PATH_WP, DOMAIN_MOODLE, PATH_MOODLE, PHP_VERSION y SERVER_ADMIN_EMAIL. Debe ejecutarse después de 04-php.sh (requiere el socket de PHP-FPM activo) y, lógicamente, después de que WordPress y Moodle ya estén desplegados en sus respectivas rutas.

Resultado final

Apache queda sirviendo tanto WordPress como Moodle mediante Virtual Hosts independientes y correctamente conectados a PHP-FPM, con el sitio por defecto desactivado, la sintaxis validada antes de aplicar cambios, y una verificación HTTP real de que ambos sitios responden.

Bloques de código más importantes

```
extraer_host() { echo "${1%%:*}"; }

extraer_puerto() {

    if [[ "$1" == *.* ]]; then echo "${1##*.}"; else echo "80"; fi

}
```

Estas dos funciones usan expansión de parámetros de bash para separar host y puerto de una misma cadena. `%%:*` elimina desde el primer ":" hasta el final (quedándose con el host), y `##*`:

elimina hasta el último ":" (quedándose con el puerto). Esto le da al script la flexibilidad de aceptar tanto una IP con puerto personalizado (modo temporal) como un dominio normal en el puerto 80 (modo definitivo), sin necesidad de dos scripts distintos.

```
<FilesMatch \.php$>

    SetHandler "proxy:unix:${SOCKET_PATH}|fcgi://localhost"

</FilesMatch>
```

Esta directiva, dentro del heredoc que genera cada Virtual Host, es la conexión real entre Apache y PHP: cualquier petición a un archivo .php se redirige mediante el módulo proxy_fcgi (habilitado en el script 02) al socket Unix de PHP-FPM (creado en el script 04), en lugar de ser procesada por Apache directamente. Esta es la arquitectura moderna recomendada para PHP, más eficiente en memoria que el clásico mod_php.

```
if [ -f "$dest_file" ]; then

    timestamp=$(date +%Y%m%d%H%M%S)

    cp "$dest_file" "${dest_file}.bak.${timestamp}" || error "..."

fi

cp "$src_file" "$dest_file" || error "..."
```

A diferencia de scripts anteriores que hacían un único respaldo .bak (sin sobrescribir si ya existía), aquí cada respaldo incluye una marca de tiempo (timestamp). Esto permite ejecutar el script



varias veces conservando un historial completo de cada configuración anterior, útil cuando se está ajustando la configuración de los Virtual Hosts de forma iterativa durante las pruebas.

LU
CEM
ASPI
CIO

Script 8: 08-security.sh

Qué hace en general

Aplica un endurecimiento de seguridad básico al servidor: instala y configura el firewall UFW con una política de "denegar todo por defecto, permitir solo lo necesario", y aplica ajustes de endurecimiento al servicio SSH (bloqueo de login root, tiempo de espera de login, puerto personalizable y, opcionalmente, desactivación de autenticación por contraseña).

Cómo funciona paso a paso

- **Configuración inicial:** Modo estricto, rutas, helpers, root. Carga config.env solo si existe (a diferencia de otros scripts, no es estrictamente obligatorio). Define valores por defecto para SSH_PORT (22) y SSH_DISABLE_PASSWORD_AUTH (false) si no vienen definidos.
- **Paso 1 — Instalación de UFW:** Verifica si ya está instalado; si no, actualiza índices e instala el paquete ufw.
- **Paso 2 — Configuración de reglas de red:** Establece las políticas por defecto (denegar todo el tráfico entrante, permitir todo el saliente). Permite explícitamente el puerto SSH ANTES de activar el firewall (para no perder el acceso remoto), permite HTTP/HTTPS (usando el perfil 'Apache Full' si está disponible, o los puertos 80/443 directamente), abre el puerto adicional de Moodle si se está usando un puerto distinto a 80, y finalmente habilita UFW de forma forzada (--force enable) para evitar el prompt de confirmación interactivo. Al final, verifica explícitamente que las reglas de SSH y web quedaron activas consultando ufw status verbose.
- **Paso 3 — Endurecimiento de SSH:** Hace un respaldo único del sshd_config original (solo si no existe ya un respaldo, para preservar siempre la configuración de fábrica). Define una función idempotente (update_ssh_config) que modifica o agrega directivas en el archivo. Aplica: PermitRootLogin no, LoginGraceTime 30, el puerto SSH personalizado si es distinto de 22, y opcionalmente PasswordAuthentication no si SSH_DISABLE_PASSWORD_AUTH está en true (si está en false, deja la autenticación por contraseña activa pero emite una advertencia explícita).
- **Paso 4 — Validación y reinicio de SSH:** Antes de reiniciar el servicio, valida la sintaxis del archivo modificado con sshd -t. Si la validación falla, restaura automáticamente el respaldo original y aborta el script, evitando dejar el servidor sin acceso remoto por un error de configuración. Si es válida, reinicia el servicio y confirma que quedó activo.

- **Paso 5 — Resumen final:** Imprime en pantalla un resumen legible de todas las políticas de seguridad aplicadas (estado de UFW, excepciones abiertas, configuración final de SSH).

Requisitos y dependencias asumidas

Debian/Ubuntu con systemd, root/sudo, helpers.sh (config.env es opcional en este script). Se recomienda ejecutarlo al final de la secuencia de despliegue, después de que Apache y los Virtual Hosts ya estén configurados, para conocer con certeza qué puertos web deben quedar abiertos.

Resultado final

El servidor queda con un firewall activo que bloquea todo el tráfico entrante no autorizado explícitamente (solo permite SSH, HTTP, HTTPS y el puerto adicional de Moodle si aplica), y con el servicio SSH endurecido: sin acceso directo como root, con tiempo de espera de login reducido, en el puerto configurado, y con la posibilidad de exigir autenticación por llave en vez de contraseña.

Bloques de código más importantes

```
ufw default deny incoming >/dev/null 2>&1 || error "..."  
  
ufw default allow outgoing >/dev/null 2>&1 || error "..."  
  
info "Configurando excepciones en firewall..."  
  
ufw allow "${SSH_PORT}/tcp" >/dev/null 2>&1 || error "..."
```

El orden de estas líneas es crítico para la seguridad y la continuidad del acceso remoto: primero se establece la política de denegar todo el tráfico entrante por defecto, y solo después se abre explícitamente el puerto SSH. Este orden se respeta ANTES de habilitar el firewall (ufw --force enable, más adelante), de modo que en ningún momento el administrador corre el riesgo de perder la conexión SSH activa mientras se aplican las reglas.

```
UFW_STATUS=$(ufw status verbose)

if ! echo "$UFW_STATUS" | grep -qE "${SSH_PORT}.*ALLOW"; then

    error "CRÍTICO: El puerto SSH (${SSH_PORT}) no aparece como ALLOW en UFW. Abortando."

fi
```

Esta verificación no se conforma con que el comando ufw allow se haya ejecutado sin errores de sintaxis: vuelve a consultar el estado real del firewall y confirma con una expresión regular que el puerto SSH efectivamente aparece como permitido. Es una comprobación de último momento antes de dar por buena una configuración que, si fallara silenciosamente, podría dejar el servidor completamente inaccesible por red.

```
if ! sshd -t >/dev/null 2>&1; then

    advertencia "La validación de sintaxis de sshd falló tras las modificaciones."

    cp "$SSHD_BACKUP" "$SSHD_CONF"

    error "Se restauró el respaldo original. Abortando para prevenir que el servidor quede inaccesible por red."

fi
```



Este es probablemente el bloque de seguridad más importante de todo el proyecto: `sshd -t` valida la sintaxis del archivo de configuración de SSH sin llegar a reiniciar el servicio. Si la validación falla, el script restaura automáticamente el respaldo original antes de detenerse, evitando el escenario de peor caso: reiniciar SSH con una configuración rota y perder el único canal de acceso remoto al servidor, lo cual solo podría solucionarse con acceso físico o consola de la nube.

LU
CEM
ASPI
CIO

09-backup.sh

Qué hace en general

Genera respaldos completos e integrales de las bases de datos (MariaDB) y de los componentes de archivos esenciales (wp-content y moodledata), aplicando además una política de retención para eliminar copias antiguas.

Cómo funciona paso a paso

- Paso 1 — Generación de contexto: Genera un identificador único basado en fecha/hora (TIMESTAMP=\$(date +%Y%m%d_%H%M%S)) y prepara la carpeta objetivo con permisos estictos 700.
- Paso 2 — Respaldos de MySQL/MariaDB: Ejecuta mysqldump con los parámetros --single-transaction --quick para WordPress y Moodle, verifica que los volcados sean válidos y los comprime con gzip.
- Paso 3 — Embalaje de archivos: Empaqueta las carpetas wp-content y moodledata mediante tar -czf de forma relativa y prueba la integridad de los archivos resultantes con tar -tzf.
- Paso 4 — Informe de utilización: Despliega el peso final en disco de cada archivo generado con la utilidad du -h.
- Paso 5 — Purga automática por retención: Utiliza el comando find con el parámetro -mtime +"\$BACKUP_RETENTION_DIAS" para localizar y eliminar archivos de respaldo que superen la cantidad de días permitidos.

Requisitos y dependencias asumidas

Acceso administrativo a MySQL/MariaDB, utilidades mysqldump, gzip, tar, find, y espacio libre suficiente en el disco.

Resultado final

Un conjunto de 4 archivos de copia de seguridad validados, garantizando la continuidad del negocio y el control del espacio en disco.

Bloques de código más importantes:

Bash

```
ARCHIVOS_A_ELIMINAR=$(find "$BACKUP_DIR" -type f -mtime +"$BACKUP_RETENTION_DIAS" \  
    \( -name "wp_db_*.sql.gz" -o -name "moodle_db_*.sql.gz" -o -name "wp_content_*.tar.gz" -o -name \  
    "moodledata_*.tar.gz" \) 2>/dev/null || true)  
  
CANTIDAD_ELIMINADOS=0  
  
if [[ -n "$ARCHIVOS_A_ELIMINAR" ]]; then  
    CANTIDAD_ELIMINADOS=$(echo "$ARCHIVOS_A_ELIMINAR" | grep -v '^$' | wc -l || echo 0)  
    echo "$ARCHIVOS_A_ELIMINAR" | xargs rm -f || error "Error al eliminar respaldos antiguos."  
  
fi
```

Implementa el ciclo de vida de los datos: evita el colapso de almacenamiento en el servidor al evaluar la antigüedad de los archivos y remover automáticamente únicamente los respaldos que hayan expirado según los días definidos en la variable `BACKUP_RETENTION_DIA`

Script 10: deploy-all.sh

Qué hace en general

Es el script orquestador de todo el proyecto: ejecuta en orden estricto la secuencia completa de despliegue (del script 01 al 08), deteniendo todo el proceso inmediatamente si alguno de los scripts individuales falla, para evitar que el despliegue continúe sobre una base incompleta o mal configurada.

Cómo funciona paso a paso

- **Configuración inicial:** Modo estricto, resolución de rutas, carga de helpers.sh y verificación de privilegios de root.
- **Definición de la secuencia:** Declara un arreglo (SCRIPTS_A_EJECUTAR) con los nombres de los ocho scripts, en el orden exacto en que deben ejecutarse: actualización del sistema, Apache, MariaDB, PHP, WordPress, Moodle, Virtual Hosts y seguridad.
- **Ejecución secuencial y aislada:** Recorre el arreglo con un bucle for, y para cada script verifica primero que el archivo exista físicamente antes de intentar ejecutarlo. Cada script se ejecuta con `bash script_path` como un subprocesso aislado.
- **Manejo de errores en cadena:** Si un script devuelve un código de salida distinto de cero, el orquestador detiene inmediatamente toda la secuencia con la función `error` (heredada de `helpers.sh`), sin continuar con los scripts restantes.
- **Mensaje final de éxito:** Si los ocho scripts se ejecutaron correctamente en orden, muestra un mensaje de confirmación de que el despliegue completo terminó con éxito.

Requisitos y dependencias asumidas

Debe ejecutarse desde el mismo directorio donde residen todos los scripts numerados (01 al 08) y `helpers.sh`, con permisos de `root/sudo`. Todos los scripts individuales deben ya tener sus propias validaciones de `config.env`, ya que este orquestador no valida variables de configuración por sí mismo, solo delega la ejecución.

Resultado final

Si todo sale bien, el servidor queda completamente desplegado de principio a fin (sistema actualizado, Apache y PHP-FPM configurados, MariaDB con sus bases de datos, WordPress y Moodle instalados, Virtual Hosts activos, y el firewall/SSH endurecidos) con una sola ejecución. Si algo falla en cualquier punto, el proceso se detiene de inmediato en ese script específico, dejando claro exactamente en qué paso ocurrió el problema.



Bloques de código más importantes

```
SCRIPTS_A_EJECUTAR=(  
  
    "01-update.sh"  
  
    "02-apache.sh"  
  
    "03-mariadb.sh"  
  
    "04-php.sh"  
  
    "05-wordpress.sh"  
  
    "06-moodle.sh"  
  
    "07-vhosts.sh"  
  
    "08-security.sh"  
  
)
```

Este arreglo es, en esencia, el mapa de dependencias de todo el proyecto expresado como una simple lista ordenada: el orden importa, ya que cada script asume que el anterior ya se ejecutó (por ejemplo, 07-vhosts.sh necesita el socket de PHP-FPM que crea 04-php.sh, y 05-wordpress.sh necesita la base de datos que crea 03-mariadb.sh). Mantener esta lista centralizada facilita agregar, quitar o reordenar módulos del despliegue en el futuro.

```
for script in "${SCRIPTS_A_EJECUTAR[@]}; do  
  
    script_path="${SCRIPT_DIR}/${script}"  
  
    if [ -f "$script_path" ]; then
```

```
info ">>> Iniciando: ${script}"

bash "$script_path" || error "El script ${script} ha fallado. Se aborta la secuencia completa."

info ">>> Finalizado: ${script}"

else

    error "Archivo no encontrado: ${script_path}. No se puede continuar el despliegue."

fi

done
```

Este es el núcleo del orquestador. Cada script se invoca con `bash script_path` (no con `source`), lo que significa que se ejecuta en un subproceso completamente aislado, con su propio entorno y su propio set -euo pipefail. El operador `|| error "..."` aprovecha el código de salida del subproceso: si el script individual termina con error (por ejemplo, porque uno de sus propios comandos falló bajo su set -e interno), el orquestador lo detecta inmediatamente y detiene toda la cadena, en lugar de continuar ciegamente con el siguiente módulo sobre una base posiblemente rota.

guardar-diseno.sh (save-desing.sh)

Qué hace en general

Extrae la configuración visual, personalizaciones de color, tema activo y registro de archivos multimedia del tema *Moove* desde una base de datos de Moodle operativa para permitir su posterior replicación.

Cómo funciona paso a paso

- **Paso 1 — Modo estricto y contexto:** Ejecuta `set -euo pipefail` y se posiciona en el directorio base para importar `helpers.sh` y `config.env`.
- **Paso 2 — Preparación del directorio objetivo:** Crea la ruta definida en `DISENO_EXPORT_DIR` y la protege con permisos 700.
- **Paso 3 — Dump de configuraciones del tema:** Vuelca la tabla `mdl_config_plugins` donde el plugin coincida con `theme_moove` mediante `mysqldump`.
- **Paso 4 — Dump del tema activo global:** Vuelca el valor del tema seleccionado en la tabla `mdl_config`.
- **Paso 5 — Catalogación de imágenes:** Consulta `mdl_files` en la base de datos para registrar los nombres de archivos gráficos en `imagenes_theme.txt`.

Requisitos y dependencias asumidas

Moodle instalado y configurado en MariaDB, permisos de lectura en la base de datos, variable `DISENO_EXPORT_DIR` declarada en `config.env`.

Resultado final

Un conjunto de archivos SQL y registros de texto con el diseño exportado, listo para integrarse al repositorio.

Bloques de código más importantes

```
Bash

exportar_tabla() {
    local tabla="$1"
    local condicion="$2"
    local destino="$3"
```

```
MYSQL_PWD="$DB_MOODLE_PASS" mysqldump \  
  
-u "$DB_MOODLE_USER" \  
  
--no-create-info \  
  
--replace \  
  
"$DB_MOODLE_NAME" "$tabla" \  
  
--where="$condicion" > "$destino" \  
  
}
```

Garantiza una exportación limpia e inocua: utiliza la variable de entorno MYSQL_PWD para evitar exponer la contraseña en el listado de procesos de Linux (ps aux), y emplea --replace para facilitar la posterior importación sin colisiones de clave primaria.

aplicar-diseno.sh (load-desing.sh)

Qué hace en general

Aplica sobre una instancia limpia de Moodle la configuración estética, colores y activación del tema *Moove* exportados previamente.

Cómo funciona paso a paso

- **Paso 1 — Validación de insumos:** Verifica que los volcados SQL (moove_config.sql y theme_activo.sql) existan y tengan contenido.
- **Paso 2 — Verificación del plugin:** Comprueba que la carpeta del tema `${PATH_MOODLE}/theme/moove` exista físicamente en el servidor.
- **Paso 3 — Importación de la base de datos:** Ejecuta la inyección SQL del archivo de configuración contra MariaDB.
- **Paso 4 — Activación oficial mediante CLI:** Utiliza la herramienta de línea de comandos de Moodle (admin/cli/cfg.php) para activar el tema *Moove*, garantizando la correcta escritura en caché.
- **Paso 5 — Purga de caché:** Invoca `purge_caches.php` para obligar a Moodle a renderizar la nueva interfaz de inmediato.

Requisitos y dependencias asumidas

Instancia de Moodle funcional, plugin `theme_moove` previamente descargado, archivos de exportación presentes.

Resultado final

Un sitio Moodle reconfigurado estéticamente de forma automatizada, quedando pendiente únicamente la subida manual de imágenes.

Bloques de código más importantes

Bash

```
# Se activa el tema con admin/cli/cfg.php en vez de importar directamente theme_activo.sql  
sudo -u www-data php "${PATH_MOODLE}/admin/cli/cfg.php" --name=theme --set=moove  
ok "Tema activo configurado como Moove"
```

```
sudo -u www-data php "${PATH_MOODLE}/admin/cli/purge_caches.php"
```

Evita inconsistencias en Moodle: usar la API de CLI propia de Moodle (cfg.php) bajo el usuario del servidor web (www-data) asegura que los cambios no queden atascados en la caché de la aplicación

restore.sh

Qué hace en general

Restaura en un servidor nuevo un respaldo previo completo (bases de datos y directorios de archivos de WordPress y Moodle) utilizando un sello de tiempo (*timestamp*).

Cómo funciona paso a paso

- **Paso 1 — Verificación del Timestamp:** Valida que el parámetro de fecha/hora sea recibido y que los 4 archivos de backup existan en `RESTORE_SOURCE_DIR`.
- **Paso 2 — Confirmación de seguridad:** Solicita al usuario ingresar la palabra "RESTAURAR" en mayúsculas para evitar sobrescribir datos por error.
- **Paso 3 — Restauración de bases de datos:** Descomprime temporalmente los volcados de WordPress y Moodle, valida que contengan instrucciones `CREATE TABLE` e inyecta la estructura en MariaDB.
- **Paso 4 — Reemplazo de archivos web:** Crea respaldos de seguridad de las carpetas actuales (wp-content y moodledata) y extrae los paquetes del backup asignando la propiedad al usuario www-data.

- **Paso 5 — Limpieza y caché:** Invoca la purga de caché de Moodle y despliega las instrucciones de verificación de URL.

Requisitos y dependencias asumidas

Servidor con la pila LAMP ya desplegada, los 4 archivos de backup (.sql.gz y .tar.gz) presentes en la ruta de restauración.

Resultado final

Un servidor clonado o migrado exactamente al estado en el que se tomó la copia de seguridad.

Bloques de código más importantes

Bash

```
mkdir -p "$MOODLE_DATA_DIR"
```

```
tar -xzf "$MOODLEDATA_TAR" -C "$MOODLE_DATA_DIR"
```

```
chown -R www-data:www-data "$MOODLE_DATA_DIR"
```

Previene fallos estructurales en Moodle: descomprime los datos respetando la jerarquía relativa de archivos e inmediatamente restablece los permisos de usuario www-data:www-data para evitar errores de permisos de escritura en la plataforma.

menu.sh

Qué hace en general

Ofrece un panel de control interactivo mediante consola para la administración integral del servidor: despliegue de la pila web, mantenimiento, ejecución de diagnósticos y gestión de backups.

Cómo funciona paso a paso

- **Paso 1 — Verificación del entorno:** Se asegura de ser ejecutado desde el directorio raíz verificando la existencia de helpers.sh y carga config.env.
- **Paso 2 — Funciones de ejecución segura:** Implementa run_script() para validar si un módulo existe en la carpeta scripts/ antes de intentar ejecutarlo.
- **Paso 3 — Diagnósticos del sistema:** check_services() analiza Apache, MariaDB y SSH; check_domains() efectúa peticiones HTTP con curl a los dominios configurados; system_info() despliega el uso de recursos.
- **Paso 4 — Gestor de logs:** logs_menu() permite inspeccionar las últimas líneas (tail -n 40) de los registros de instalación y errores de Apache.
- **Paso 5 — Bucle interactivo:** Despliega una interfaz gráfica en texto dentro de un bucle while true, leyendo la opción elegida por el operador.

Requisitos y dependencias asumidas

Servicios de sistema (systemctl), utilidades de diagnóstico (curl, awk, df, free), helpers.sh y config.env.

Resultado final

Un centro de mandos administrativo centralizado que abstrae la complejidad de la consola para los administradores.

Bloques de código más importantes

```
Bash
check_domains() {
    echo ""
    info "Verificando acceso a los dominios:"
    local domains=("${DOMAIN_WP:-www.unahconecta.com}" "${DOMAIN_MOODLE:-moodle.unahconecta.com}")
    for url in "${domains[@]}; do
        code=$(curl -s -o /dev/null -m 5 -w "%{http_code}" "http://${url}" 2>/dev/null || echo "000")
        if [[ "$code" == "200" ]]; then
            ok "http://${url} responde correctamente (HTTP ${code})"
        elif [[ "$code" == "000" ]]; then
            advertencia "http://${url} no responde (sin conexión o DNS no resuelto)"
        else
            advertencia "http://${url} respondió con HTTP ${code}"
        fi
    done
}
```

Realiza una comprobación en tiempo real del estado web mediante código HTTP, ofreciendo una validación rápida del funcionamiento global de la infraestructura.

helpers.sh

Qué hace en general

Proporciona una librería auxiliar de funciones reutilizables y códigos de colores ANSI para estandarizar las salidas por consola, mensajes de estado (éxito, advertencia, error) y validaciones de permisos dentro del proyecto.

Cómo funciona paso a paso

- **Paso 1 — Definición de paleta ANSI:** Almacena códigos de escape para formatear texto en verde, rojo, amarillo y azul.
- **Paso 2 — Cálculo de progresión:** Analiza el directorio ./scripts para contar la cantidad de archivos y determinar dinámicamente el número total de pasos.
- **Paso 3 — Funciones de retroalimentación:** Implementa paso(), ok(), advertencia(), e info() para imprimir salidas limpias y formateadas.
- **Paso 4 — Control de errores estricto:** La función error() imprime un fallo formateado e interrumpe de inmediato la ejecución con exit 1.
- **Paso 5 — Validación de superusuario:** require_root() comprueba si el EUID es igual a 0.

Requisitos y dependencias asumidas

Entorno Bash con soporte para secuencias de escape ANSI.

Resultado final

Interfaz de consola unificada, legible y protegida contra ejecuciones sin permisos administrativos.

Bloques de código más importantes

```
Bash
# Verifica que el script corra como root/sudo
```

```
require_root() {  
  if [[ $EUID -ne 0 ]]; then  
    error "Este script debe ejecutarse con sudo o como root."  
  fi  
}
```

Es la barrera de seguridad del framework: impide que cualquier proceso de instalación o modificación del sistema se ejecute sin los privilegios requeridos, interrumpiendo el flujo antes de causar inconsistencias.

Archivo Importante del sistema

config.env

Qué hace en general

Actúa como la fuente centralizada de verdad y configuración para todo el proyecto UNAH-CONECTA, definiendo dominios, rutas del sistema de archivos, versiones, credenciales de bases de datos y parámetros de los módulos WordPress y Moodle.

Cómo funciona paso a paso

- **Paso 1 — Parámetros de Red y Dominio:** Define las URLs de acceso principal (www.unahconecta.com y moodle.unahconecta.com).
- **Paso 2 — Rutas del Sistema:** Asigna los directorios donde residen las plataformas web, los archivos de respaldo y el repositorio de automatización.
- **Paso 3 — Variables de MariaDB:** Almacena los nombres de bases de datos, usuarios y claves tanto para WordPress como para Moodle.
- **Paso 4 — Credenciales e Instalación Web:** Especifica usuarios administradores, correos de contacto, versiones de software (PHP 8.3) y ramas estables de Moodle (MOODLE_405_STABLE).
- **Paso 5 — Políticas de Mantenimiento:** Define la vigencia de los archivos de backup (BACKUP_RETENTION_DIAS="7").

Requisitos y dependencias asumidas

Formato compatible con Bash (CLAVE="VALOR"), destinado a ser importado mediante source config.env.

Resultado final



Variables globales cargadas y disponibles para automatizar cualquier script sin necesidad de duplicar datos ni hardcodear valores.

Bloques de código más importantes

```
Bash
# --- Base de datos ---
DB_WP_NAME="wordpress_db"
DB_WP_USER="wp_user"
DB_WP_PASS="WordPress1234*" # <-- Cambiar antes del despliegue

DB_MOODLE_NAME="moodle_db"
DB_MOODLE_USER="moodle_user"
DB_MOODLE_PASS="moodle1234*" # <-- Cambiar antes del despliegue
```

Define la capa de persistencia de datos centralizada; cambiar estas variables aquí propaga las credenciales a los scripts de instalación, mantenimiento y restauración automáticamente.

Backups

1. Generalidades del Sistema de RespalDOS

El sistema de copias de seguridad de **UNAH-CONECTA** está completamente automatizado para garantizar la integridad y continuidad de los servicios de WordPress y Moodle.

El modelo de respaldos abarca dos capas principales:

1. **Bases de Datos (MariaDB):** Volcados comprimidos en .sql.gz utilizando la herramienta mysqldump.
2. **Archivos del Sistema:** Empaquetados en .tar.gz que resguardan los directorios dinámicos wp-content (WordPress) y moodledata (Moodle).

2. Procedimiento de Creación de RespalDOS

El proceso de copia de seguridad puede ejecutarse de forma manual desde el menú de administración o directamente mediante línea de comandos.

Opción A: Ejecución mediante el Menú Principal

1. Accede al servidor vía SSH.
2. Ejecute el panel interactivo:
sudo ./menu.sh

3. Seleccione la opción 7) **Ejecutar backups completo.**
4. El script generará los archivos correspondientes en el directorio configurado (/var/backups/unahconecta o la ruta definida en config.env).

Opción B: Ejecución Directa del Script (09-backup.sh)

Para ejecutar el respaldo de forma independiente o a través de tareas programadas (cron):

Bash

```
sudo ./scripts/09-backup.sh
```

Flujo Interno del Procedimiento:

- **Generación de Timestamp:** El script genera un identificador único basado en la fecha y hora de ejecución (ej. 20260730_194000).
- **Resguardo de MariaDB:** Realiza la extracción de las bases de datos de WordPress (wordpress_db) y Moodle (moodle_db) verificando la integridad de las tablas.
- **Resguardo de Archivos:** Crea los paquetes de datos para wp-content y moodledata.
- **Aplicación de Políticas de Retención:** Revisa la antigüedad de las copias existentes en el directorio de backups y elimina automáticamente los archivos que superen el límite de días establecido en la variable BACKUP_RETENTION_DIAS de config.env.

3. Procedimiento de Restauración del Sistema

El script restore.sh permite recuperar un estado previo del servidor utilizando el *timestamp* asignado a un grupo de respaldos.

Pasos para la Restauración:

1. **Identificar el Timestamp:** Consulte los archivos disponibles dentro de la carpeta de respaldos para copiar el identificador común de los 4 archivos (wp_db_TIMESTAMP.sql.gz, moodle_db_TIMESTAMP.sql.gz, wp_content_TIMESTAMP.tar.gz, moodledata_TIMESTAMP.tar.gz).
2. **Ejecutar el Script de Restauración:**

Bash

```
sudo ./scripts/restore.sh <TIMESTAMP>
```



3. **Confirmación de Seguridad:** Para evitar la sobreescritura accidental de datos en producción, el sistema solicitará escribir explícitamente la palabra RESTAURAR en mayúsculas antes de continuar.
4. **Validación Automática:**
 - Descomprime e inyecta la estructura SQL en las bases de datos de MariaDB.
 - Crea respaldos de seguridad preventivos de los directorios actuales.
 - Restaura los archivos web y restablece de forma automática los permisos de propiedad al usuario del servidor web (www-data:www-data).
 - Limpia las cachés internas de Moodle mediante la interfaz CLI (purge_caches.php).

4. Gestión y Exportación del Diseño (Tema Moove)

Adicionalmente a los respaldos globales, el sistema cuenta con procedimientos específicos para congelar y migrar la capa estética de Moodle.

1. Exportar la Configuración Visual (13-guardar-diseno.sh)

Extrae únicamente la parametrización de colores, estilos y temas activos de Moodle para versionarlos en el proyecto:

Bash

```
sudo ./scripts/guardar-diseno.sh
```

- Genera los volcados moove_config.sql, theme_activo.sql y el listado de recursos gráficos en la carpeta de exportación del diseño.

2. Aplicar la Configuración Visual (14-aplicar-diseno.sh)

Importa el diseño preconfigurado sobre una instalación de Moodle existente:

Bash

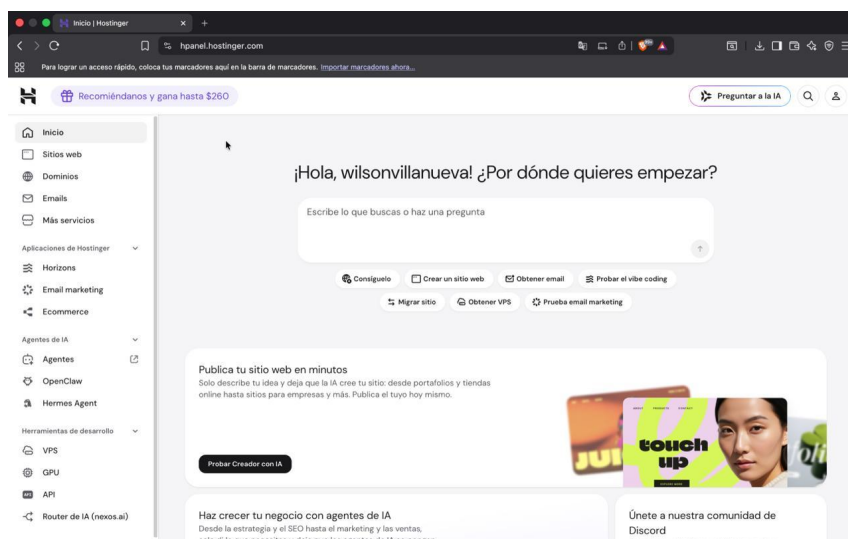
```
sudo ./scripts/aplicar-diseno.sh
```

- Inyecta las variables de estilo en la base de datos, fuerza la activación del tema Moove mediante admin/cli/cfg.php y purga las cachés del sitio.

Aquí tienes la sección redactada con el mismo estilo formal y técnico que venimos manejando para la documentación del proyecto en Word. Se detallan de forma técnica los valores exactos mostrados en las capturas de pantalla de Hostinger (dirección IP, registro CNAME, valores TTL) y se incluyen los marcadores de posición para que insertes las imágenes correspondientes en orden.

Configuración de Dominio y Resolución de IP

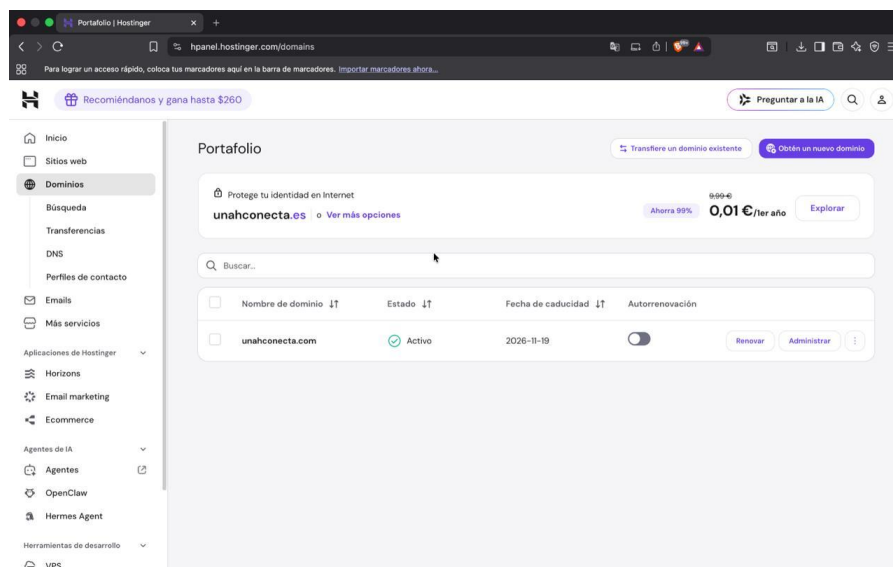
Para la gestión y resolución del dominio institucional unahconecta.com, se utiliza la plataforma **Hostinger** como proveedor del servicio de zona DNS. A continuación, se describe detalladamente el procedimiento técnico para enlazar el dominio registrado con la dirección IP pública de nuestro servidor web.



Paso 1: Acceso al panel de Hostinger (hPanel)

Para iniciar el proceso de vinculación, es necesario ingresar al panel de control central (hPanel) de Hostinger utilizando las credenciales administrativas de la cuenta donde reside la propiedad del dominio unahconecta.com.

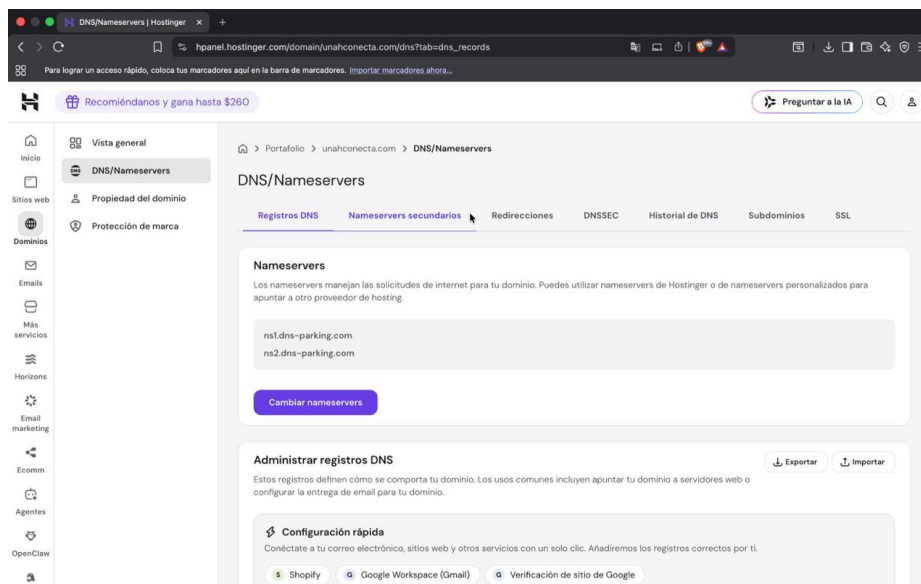
Una vez dentro de la pantalla de bienvenida del hPanel, en el menú lateral izquierdo seleccionamos la opción **Dominios** para acceder al catálogo global de zonas contratadas.



Paso 2: Ubicación de la sección de DNS del dominio

Dentro de la sección de **Portafolio / Dominios**, se ubica la lista de dominios asociados a la cuenta. Identificamos el dominio principal unahconecta.com (el cual debe figurar con estado *Activo*) y hacemos clic en el botón **Administrar** o seleccionamos del menú lateral la pestaña **DNS / Nameservers**.

Esta sección es fundamental ya que nos permite gestionar los servidores de nombres y manipular directamente la tabla de registros que definen la ruta del tráfico de red hacia la infraestructura del servidor.

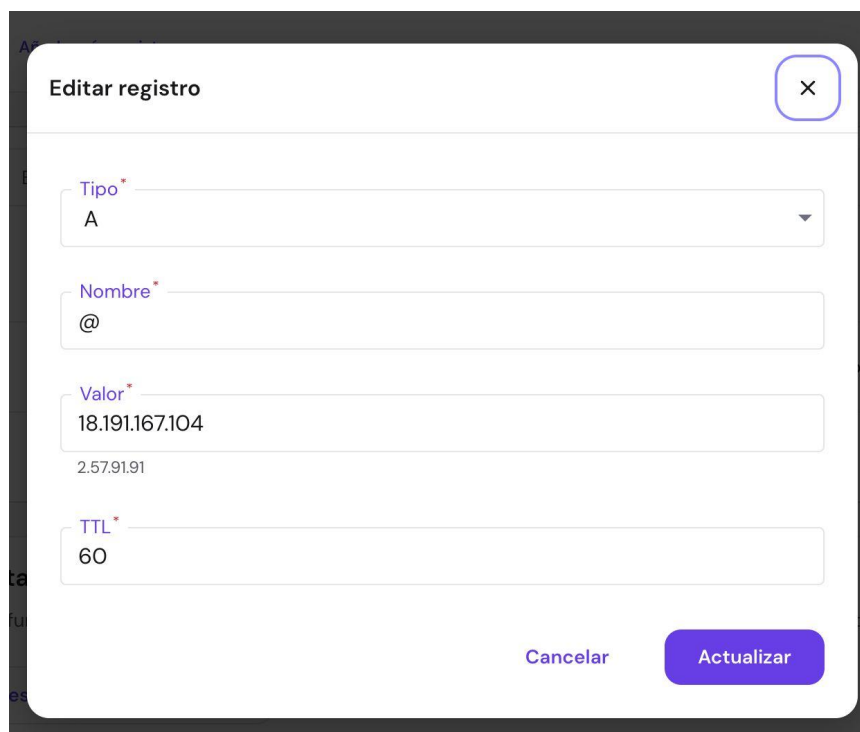


Paso 3:

Configuración del Registro de tipo A

Una vez situados en la pestaña **Registros DNS** dentro del módulo de administración de DNS/Nameservers:

1. Ubicamos la tabla de registros DNS o hacemos clic sobre el registro de tipo **A** cuyo nombre (*Host*) sea **@**.
2. Se despliega la ventana modal **Editar registro**, donde asignamos los siguientes parámetros exactos:
 - **Tipo:** A
 - **Nombre:** @ (*representa el raíz del dominio unahconecta.com*)
 - **Valor (IP):** 18.191.167.104 (*IP pública correspondiente al servidor donde se aloja el proyecto*)
 - **TTL:** 60 (*Time To Live en segundos, configurado bajo para permitir una propagación rápida de los cambios de IP*)
3. Hacemos clic en el botón **Actualizar** para aplicar los cambios en la zona.



Paso 4: Verificación y ajuste del Registro CNAME

Para garantizar que los usuarios puedan acceder al portal tanto escribiendo la URL limpia (unahconecta.com) como incluyendo el alias web (www.unahconecta.com), procedemos con la edición del registro **CNAME**:

1. En la modal de edición configuramos los siguientes datos:
 - **Tipo:** CNAME
 - **Nombre:** www
 - **Objetivo:** unahconecta.com
 - **TTL:** 300
2. Confirmamos haciendo clic en **Actualizar**. Esto crea un alias canónico que redirige todo el tráfico enviado al subdominio www directamente hacia la entrada principal configurada en el Registro A.

Editar registro

Tipo*

CNAME

Nombre*

www

Objetivo*

unahconecta.com

example.com

TTL*

300

Cancelar

Actualizar

Paso 5: Guardado, tabla final y verificación de resolución

Tras realizar la actualización de los registros individuales, la tabla principal de **Administrar registros DNS** refleja la estructura operativa final de la zona:

Tipo	Nombre	Prioridad	Contenido / Objetivo	TTL
CNAME	www	0	unahconecta.com	300
A	@	0	18.191.167.104	60

Al guardar y aplicar estas directivas en Hostinger, las peticiones HTTP/HTTPS entrantes hacia unahconecta.com y www.unahconecta.com son resueltas de manera exitosa apuntando directamente hacia la dirección IP de nuestro servidor Linux web.



Inicio

Sitios web

Dominios

Emails

Más servicios

Horizons

Email marketing

Ecomm

Agentes

OpenClaw

Recomiéndanos y gana hasta \$260

Preguntar a la IA

Vista general

DNS/Nameservers

Propiedad del dominio

Protección de marca

Elige tipo

-

-

14400

Añadir registro

+ Añade más registros

Q Buscar

☐	Tipo ↓↑	Nombre ↓↑	Prioridad ↓↑	Contenido ↓↑	TTL ↓↑	
☐	CNAME	www	0	unahconecta.com	300	
☐	A	@	0	18.191.167.104	60	

Restablecer registros DNS

Esta función restablece todos los registros DNS existentes de unahconecta.com a los valores predeterminados.

Restablecer registros DNS



PROBLEMAS Y SOLUCIONES

Durante el proceso de instalación, configuración y puesta en marcha del servidor Ubuntu, se presentaron diversas incidencias técnicas. En esta sección se documentan los problemas más significativos encontrados, junto con su causa raíz y la solución técnica implementada.

Incidencia 1: Error 403 Forbidden o Permisos Denegados al acceder a WordPress / Moodle

- **Descripción del Problema:** Al intentar ingresar a los sitios web configurados a través del navegador, Apache devolvía un error 403 Forbidden o las aplicaciones no podían escribir archivos de configuración temporales.
- **Causa Raíz:** Los archivos y directorios extraídos en /var/www/html/ pertenecían al usuario root con permisos restrictivos (700), impidiendo que el demonio del servidor web (www-data) pudiera leer o ejecutar los archivos.
- **Solución Aplicada:** Se ajustó la propiedad y los permisos recursivos de las carpetas de las aplicaciones web mediante los comandos:

Bash

```
sudo chown -R www-data:www-data /var/www/html/  
sudo find /var/www/html/ -type d -exec chmod 755 {} \  
sudo find /var/www/html/ -type f -exec chmod 644 {} \;
```

Incidencia 2: Moodle rechaza el directorio de datos (moodledata)

- **Descripción del Problema:** Durante la ejecución del script de instalación de Moodle (06-moodle.sh), el asistente devolvía un mensaje de error indicando que la carpeta de datos no era segura o no tenía permisos de escritura.
- **Causa Raíz:** Por razones de seguridad, Moodle exige que la carpeta moodledata esté ubicada **fuera** del directorio público del servidor web (/var/www/html/) y posea permisos de lectura/escritura completos para el usuario www-data.
- **Solución Aplicada:** Se reubicó la carpeta a /var/moodledata/ y se le asignaron los permisos adecuados:

Bash

```
sudo mkdir -p /var/moodledata  
sudo chown -R www-data:www-data /var/moodledata  
sudo chmod -R 777 /var/moodledata
```

Incidencia 3: Fallo de conexión a MariaDB en la instalación de WordPress



- **Descripción del Problema:** WordPress mostraba el mensaje *"Error al establecer una conexión con la base de datos"* al ejecutar el asistente web de configuración.
- **Causa Raíz:** Las credenciales de la base de datos en el archivo wp-config.php no coincidían con el usuario y la contraseña creados en MariaDB, o el usuario no tenía asignados los privilegios globales sobre la base de datos wordpress_db.
- **Solución Aplicada:** Se ingresó a la consola de MariaDB y se volvieron a otorgar los privilegios correspondientes:

SQL

```
GRANT ALL PRIVILEGES ON wordpress_db.* TO 'wp_user'@'localhost' IDENTIFIED BY  
'PasswordSegura123!';  
FLUSH PRIVILEGES;
```

Incidencia 4: Bloqueo de acceso a Webmin y FTP por el Cortafuegos UFW

- **Descripción del Problema:** Al activar el cortafuegos mediante 08-security.sh, se perdió inmediatamente el acceso a la interfaz web de Webmin (puerto 10000) y las conexiones FTP del cliente se quedaban en estado colgado.
- **Causa Raíz:** La política predeterminada del cortafuegos (UFW) estaba configurada en denegar todo el tráfico entrante (*Default Deny*), y los puertos 10000 (Webmin) y 20/21 (FTP) no habían sido habilitados explícitamente previo a la activación.
- **Solución Aplicada:** Se añadieron las reglas correspondientes en UFW y se recargó el servicio:

Bash

```
sudo ufw allow 10000/tcp  
sudo ufw allow 21/tcp  
sudo ufw allow 20/tcp  
sudo ufw reload
```



CONCLUSIONES

Basado en el desarrollo, pruebas e implementación de la infraestructura sobre **Ubuntu Server**, se presentan las siguientes conclusiones técnicas:

1. **Eficiencia en la Automatización mediante Bash:** La implementación de un flujo modular de 11 scripts en Bash redujo significativamente el tiempo de despliegue del servidor y eliminó el margen de error humano asociado a la configuración manual de servicios complejos como LAMP, FTP, WordPress y Moodle.
2. **Robustez del Stack LAMP:** La combinación de Apache, MariaDB y PHP sobre Ubuntu Server demostró ser un entorno sumamente estable, capaz de gestionar múltiples plataformas dinámicas de forma concurrente mediante la correcta utilización de Virtual Hosts.
3. **Seguridad Multi-capa:** La integración conjunta de UFW para el filtrado de tráfico por red, el uso de SFTP con entornos enjaulados (*chroot*) para la transferencia de archivos y el endurecimiento de SSH garantizan una postura de seguridad sólida frente a accesos no autorizados.
4. **Viabilidad de Gestión Centralizada:** La incorporación de herramientas como Webmin permite a los administradores del sistema monitorear el consumo de hardware (CPU, RAM, Disco) y controlar servicios sin depender exclusivamente de la interfaz de línea de comandos, optimizando las labores de mantenimiento diario.



BIBLIOGRAFÍA

(Las siguientes referencias bibliográficas siguen el estándar de la norma APA 7.^a edición):

- Canonical Ltd. (2024). *Ubuntu Server Documentation*. Ubuntu Documentation. <https://help.ubuntu.com/lts/serverguide/>
- Free Software Foundation. (2022). *Bash Reference Manual (Version 5.2)*. GNU Project. <https://www.gnu.org/software/bash/manual/>
- Foundation for Learning Management Systems. (2023). *Moodle Administration Documentation*. Moodle Docs. <https://docs.moodle.org/>
- MariaDB Foundation. (2024). *MariaDB Knowledge Base: MariaDB Server Documentation*. MariaDB. <https://mariadb.com/kb/en/documentation/>
- The Apache Software Foundation. (2023). *Apache HTTP Server Version 2.4 Documentation*. Apache Documentation. <https://httpd.apache.org/docs/2.4/>
- WordPress.org. (2024). *WordPress Code Reference & System Requirements*. WordPress Developer Resources. <https://developer.wordpress.org/>